A hands-on tutorial on R and RStudio

A tutorial that might get used for lots of stuff

Tiago A. Marques

2026-10-07

Before you start

Where is this tutorial?

You might be reading this document as a pdf, a word or an html. These files are created by compiling (i.e. knitting) a quarto dynamic report (see many more details about dynamic reports below). The .qmd "TAMsIntro2RviaRStudioTutorial.qmd", which after compilation, creates these 3 different formats, is available in the github repository located at:

https://github.com/TiagoAMarques/AnIntro2RTutorial

Tutorial versions

This .qmd was last compiled on 2026-10-07 09:26:47.565295. The latest version of this tutorial is always available at the github repository.

Tips for non-UK native system users

You should avoid file names with anything other than small English-language letters as characters, including Latin characters like "é", "ã", "ô" or "ç", say. Also avoid spaces in file names. In short, make them as short and easily readable as possible.

Also, you should be aware that some directories names can cause issues. If you are a user with those characters in the name, and you are working via a user with your name, then this applies to you.

Finally, be careful about some directories not having necessarily the name (for the system) that Windows shows you to have (examples for PT users: "Utilizador" vs "User" and "desktop" vs "Ambiente de trabalho").

Software requirements

This tutorial assumes that R and RStudio have been previously installed in the computer you are using. The latest version of both software packages is recommended. Both are free and open source. You can get R here and Rstudio here.

It is fundamental that the difference between these two pieces of software is clear in your mind from the start:

1. R is the workhorse that does all the work in the background,
2. RStudio is just a user interface that allows you to access all the power of R in a convenient way.

Additional R packages will be later installed, but you will learn about that in due time :)

Introduction

This tutorial was created as a gentle introduction to the R environment via the RStudio interface to R.

While it could be a general introduction to R, the primary objective of this document is to serve as a "hands-on-tutorial" for courses delivered by me (TAM). I use it for both Ecologia Numérica and Modelação Ecológica, courses for biology undergraduate and MSc students, respectively, at the Department of Biology (DB) at Faculdade de Ciências da Universidade de Lisboa (FCUL).

This material does not assume any previous knowledge about R, but some basic logic and programming notions would be desirable.

To facilitate the interaction with R we leverage on RStudio, a graphical user interface (GUI), a piece of software which allows users to have at a click's distance many useful features of R. In the following sections of the tutorial you will be guided through a first session of R via RStudio, and then you will start working on a dynamic report.

The tutorial is intended to follow a presentation about R and RStudio, their interaction and capabilities ("AQuikIntro2RandRStudioInQuarto.html"). This document is also in the git repository, but the presentation itself is also hosted at RPubs: A Quick Introduction to R and RStudio where you can view it immediately.

If you are using this tutorial in class with me/a teacher, after you are exposed to the slides, you might want to jump straight to Working through R via a dynamic report. Do come back after class to all the material in between here and there, as it might contain useful context and additional relevant material. Remember, here you learn as much as you want to, and those that want the most will certainly do better.

Disclaimer: in this document I copy paste freely from other documents that I have written. Therefore, if you have read these words elsewhere by me, though luck. I claim the right to plagiarize myself here!

Preliminaries about R

There is an extensive community revolving around R, and abundant courses, tutorials, books, blogs, list servers, etc, are freely available online.

R might seem frightening at first, but even monsters can make something look more pleasant if you look from the right angle. It is all a matter of perspective :) So I will use the help of some monsters here to convince you that this is the right thing to do!

The amazing images in this document are all by Allison Horst, Artwork by '@'allison_horst, and I recommend you visit Allison's github repository filled up with amazing stats and maths illustrations, including so many amazing resources to make R look less frightening. To be honest, this section is actually also an homage to Allison's outstanding work.



Figure 1: Illustrating R Monsters: Artwork by '@'allison_horst

And it is not just about stats. If you do not understand how to find the derivative of a function after looking at Artwork by Allison Horst and her amazing visualization series on the topic, take it as a sign: just give it up, as I suspect you will never will!

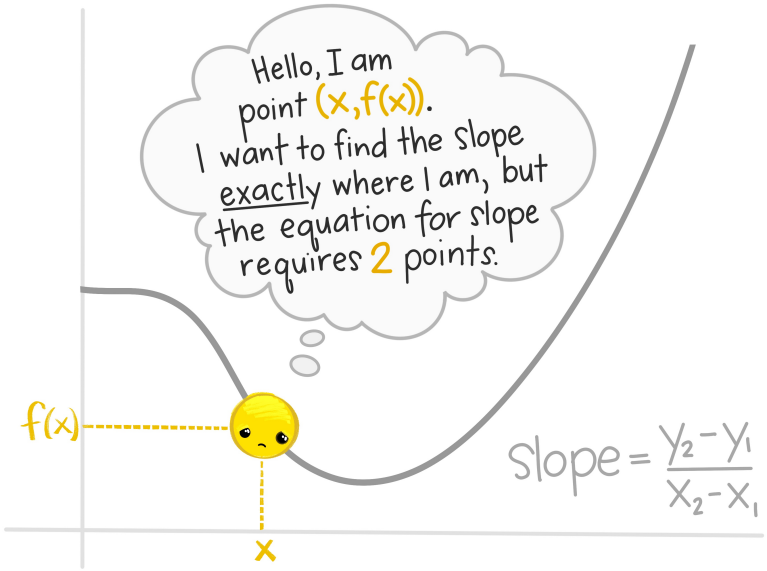


Figure 2: Illustrating a Derivative: Artwork by '@'allison_horst

Nowadays learning R by example is easy to do, with so many free online resources available to do so.

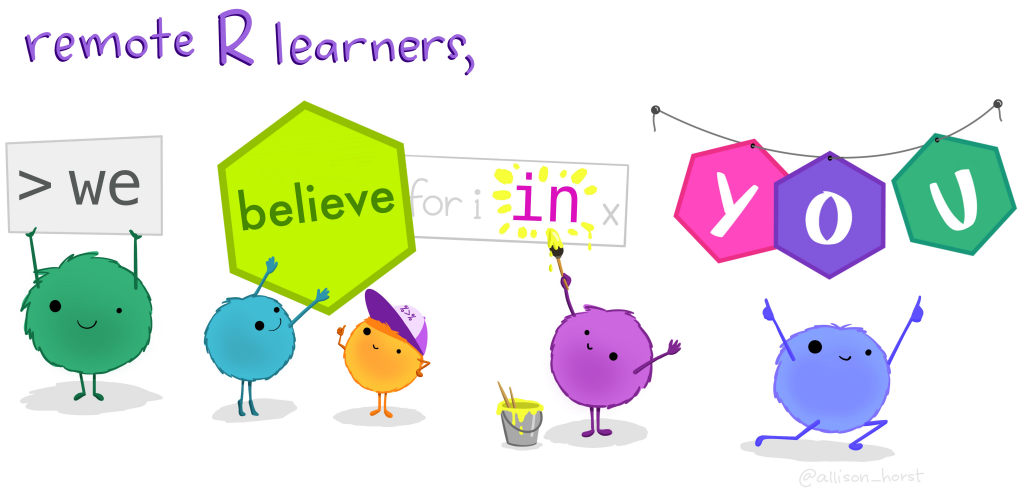


Figure 3: Illustrating learning R online: Artwork by '@'allison_horst

I recommend that you do it via the RStudio environment, since it provides an integrated environment to integrate with all R things. And there are many! And if you do so, I can guarantee that in no time you will be having fRun.



Figure 4: Illustrating having funR: Artwork by '@'allison_horst

The advantages of mastering R are priceless, but the learning curve can be daunting at first.

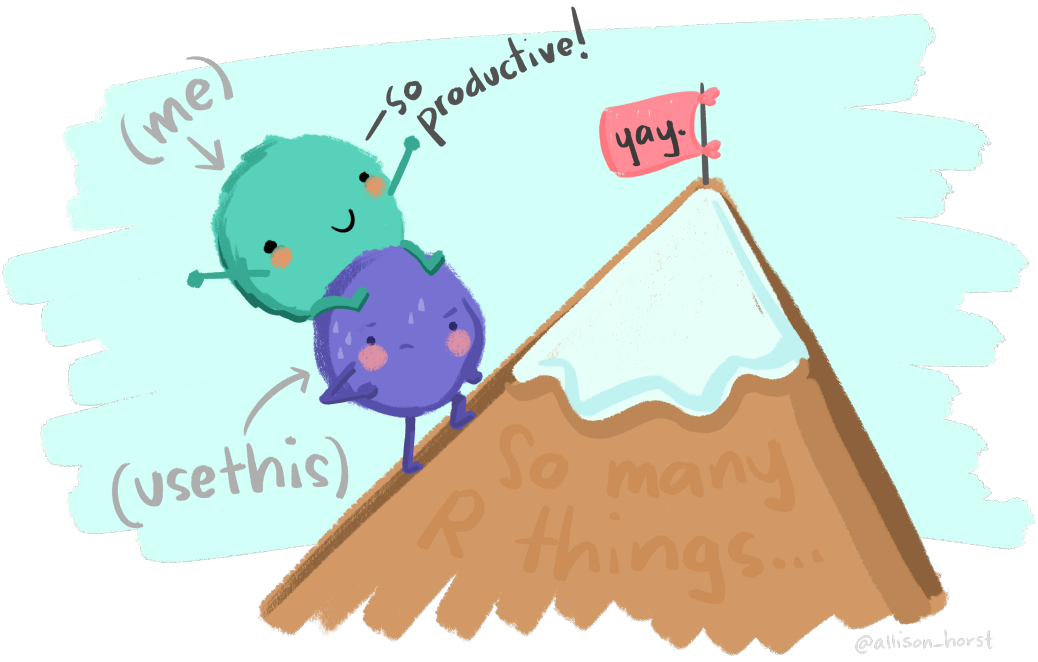


Figure 5: Illustrating R's learning curve: Artwork by '@'allison_horst

This document is written in Quarto, a tool that allows you to build dynamic reports based on R code, providing integrated documents that contain all that is required for a given project, from reading the data in to final results and discussion, passing through all the analysis and results. If you want a gentle introduction to Quarto using a hands on tutorial based on a versatile template that will do many of the things you will need to get started, look for no more, there is one here:

https://github.com/TiagoAMarques/template_4_quarto_report

Go out and explore, little grasshopper. You will conquer many great things if you do. You will become a code giant one day. But never forget, you need to be thankful to an entire community, and you are standing on the shoulders of giants!



Figure 6: Illustrating standing on the shoulders of giants: Artwork by '@'allison_horst

External R resources that might be helpful

We provide here a small list of these that might be particularly helpful for beginners:

- R webpage - the main R webpage, including links to downloading R, manuals, tutorials, dedicated search engines, etc.

- Swirl - if you want to learn R interactively from the command line, you might want to try this package. Your own R tutor at your fingertips. Try it!
- R video tutorials - video how to's in R
- Online tutorial - a course with interactive exercises
- Online course - notes for a two-day course in R
- Reference card - A very handy list of useful R functions
- Short reference card - A longer reference card with most commonly used R functions
- Cheat sheets - an incredible useful set of resources from the RStudio team, where self contained subject specific sets of functions are provided for different common tasks. Because of their importance for our work, we highlight a few here:
  - Base R: the basics in base R
  - quarto: making dynamic reports in quarto
  - lubridate: manipulating and formating dates and times
  - ggplot: plotting data
  - dplyr: manipulating data
  - sf: manipulating spatial objects

Additionally, a large number of contributed cheat sheets also exist.

At the landing page of the github repository hosting this tutorial there is a longer list of less introductory/general resources on R that might just have what you were looking for. Disclaimer: said list corresponds to a random non-exhaustive list of resources I have read and were useful to me at some point. I make no claims they might be useful to you :)

Base R vs the tidyverse

There are essentially two paradigms to coding in R. They are often as two religions, in more than one respect, so try not to get caught in the middle of a religious battle between them. On the developers own words, "The tidyverse is an **opinionated** collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures". I am R user pragmatic, I take whatever works for me in any given scenario. I was educated in base R, but the tidyverse can be quite useful.

The `tidyverse` way can be implemented via a suite of R packages, and you can read all about it in this book.



Using one or the other will lead to sometimes inconsistencies. As an example, reading data into R using base R will in general create data.frames, but reading it using `dplyr` will create `tibbles`. The way to manipulate these objects will differ too.

In general, things arguably look much better in the `tidyverse`, a poster child example being plots made in base R vs the `tidyverse ggplot2` package.

This tutorial was born in base R, but over time I am making it agnostic.

Introduction to RStudio

Typically, if I am using this tutorial in a class room, the student will have been exposed to the presentation at `AQuikIntro2RandRStudioInQuarto.html`. If you are not in a class room, you might want to take a look at it. This is also available at the repository right about here. This is an html file which you cannot visualize in github, so do download it first to see it locally in your browser. Or, just see it at A Quick Introduction to R and RStudio.

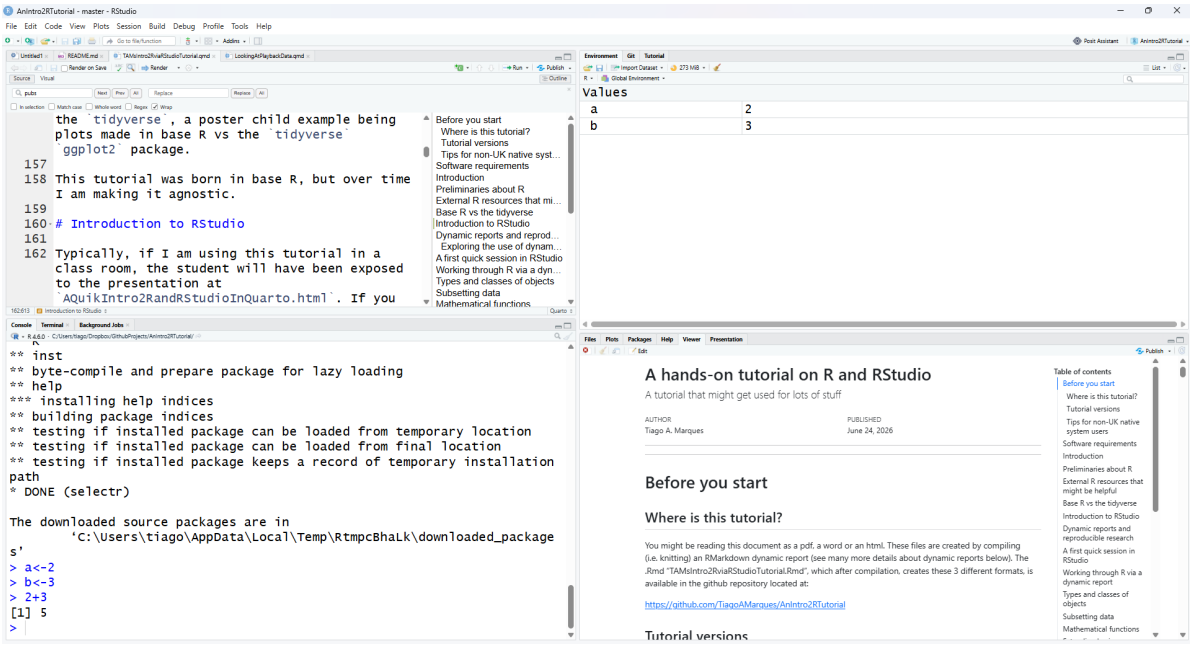
Nowadays most users (except perhaps die hard command line users) will use some sort of graphical user interface (GUI) to R. While the basic R installation comes with a simple GUI, here we adopt the use of RStudio, which considerably facilitates an introduction to R by providing many shortcuts and convenient features which we introduce next.

A major advantage of RStudio is that it makes it easy for you to type your R code into a script window, which you can easily save, and then send individual lines or blocks of code to the R command line to be acted upon. This way, you have a record of what you have done, in the saved script file, and can easily reproduce it any time you like. We strongly recommend that you save your code script.

Given RStudio has been installed, when you double-click on a R workspace it should open in RStudio. Note that, if this fails, you might have to first associate .Rdata files with RStudio. After the presentation on R and RStudio you just sat through, from within RStudio you should be able to know where to find:

- the command line (bottom left pane [1])
- the code scripts (top left pane)
- the workspace objects (top right pane)
- the loaded packages and how to load them (bottom right pane)
- the created plots (bottom right pane)
- the help files (bottom right pane)
- a file navigator system akin to windows explorer (bottom right pane)

An example screenshot of RStudio is below for reference. If you are not seeing something like this, maybe you are not in RStudio ;)



Note that you can customize the aspect of RStudio (e.g. font size and colors of the smart syntax highlighting scheme) via "Tools|Global options".

[1] All the tab positions are the RStudio defaults, but this can be customized by the user later.

A very handy feature of RStudio is that you can preview the possible arguments of functions, as well as their description, directly when you are inserting the code. Let's try doing that. Type say `seq()` in the command line or the script window and then place the cursor between the parenthesis and press the "Tab" key… Is this a nice feature or what?

Now we have met RStudio and we know how it can make our life simpler, lets move on.

Dynamic reports and reproducible research

One of the most amazing features of the integration of R and RStudio is how simple it becomes to work with dynamic reports, built on Quarto (if you are an old geeser… Quarto is like RMarkdown, on steroids - with no side effects). This will take you to the next level in data analysis. Actually, this document was itself created as a dynamic report, using Quarto. You should explore some of the basics of R quarto, and you can do so here: https://quarto.org/docs/get-started/hello/rstudio.html.

Experiment yourself to create one. In RStudio, select `File - > New file ->` ?, then just add a title, something like "My first dynamic report" and see what happens. Explore the content of the file just created and see what happens when you press the RStudio button `knit`. Experiment with the created document to try and change some of the output. Experiment in creating output as an `.html`, as a `.pdf`, and even as a `.docx` Word document.

Actually, a good way to learn and get up and running fast in Quarto is by example. Hence, I have prepared a template that you can use to create without effort a nice dynamic report. Feel free to explore the material here:

https://github.com/TiagoAMarques/template_4_quarto_report

Just download all the files into a folder, `knit` the file `QuartoTemplate.Rmd` and off you go.

Imagine the potential when you are analyzing real data, and the data changes after your report is written!

Exploring the use of dynamic reports

This section contains a separate activity that illustrates by example the usefulness of dynamic reports.

If you are working on this report in class, this might actually only be done in a separate class, after you have progressed further in the tutorial. Therefore, your teacher will let you know when it is the time to work on this task. It is also possible that your teacher will have shown you this or another similar example in class. If that is the case, you could just use this material later when revising the concepts.

Check in folder `ExampleDataAnalysis` at

https://github.com/TiagoAMarques/AnIntro2RTutorial

1. Download the file `ExampleDataAnalysis.Rmd`
2. Open `ExampleDataAnalysis.Rmd`
3. Compile (i.e. knit) `ExampleDataAnalysis.Rmd`
4. Explore the (`.html`) output of the analysis
5. Implement the last few actions described in the final "Your task" section of the document (which will lead to the creation of a new `.Rmd` file, `ExampleDataAnalysis2.Rmd`)
6. Recompile the just created `ExampleDataAnalysis2.Rmd`
7. Compare the results obtained

Describe

1. what you have learned, and
2. an hypothetical example of when this might come in handy in your work.

A first quick session in RStudio

Here we present a brief introduction to R inside RStudio, using a code script and the command line.

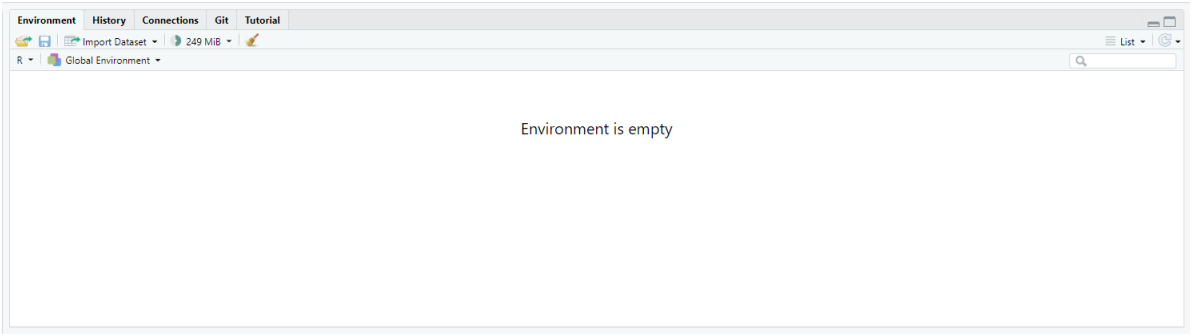
In the coming sections we will mostly consider analysis using dynamic reports via Quarto documents (.qmd), but if this is the first time you use R, it is useful to start with a session where you can see objects being created in the global environment via the command line.

Open RStudio. By default an empty workspace should appear. A workspace is like your table in a library. You can work with all the objects in your table. If you have an existing workspace, you can open it by selecting `File|Open File`. We recommend that you begin by creating a new script file (RStudio Shortcut: `Ctrl+Shift+N`; this creates a .R file) and use that file to save, and comment, all the code that you will create and execut during this current R session. In this way you will have a record of everything you did, and you can save that file as say `Session1.R` for later use.

You know that R is ready to receive a command when you see the R prompt on the command line (on the bottom left tab by default in RStudio): `>`. If you type a line of code that is not complete, R presents the `+` character, so that you, the R user, know R expects the conclusion of the current line.

Important note: while the prompt `>` and `+` might not be shown in this tutorial's code, they are often present in material online. You should not try to add either `>` nor `+` to the command line: this is something that R does for you and R will complain with an error message if you try to do it yourself! Past experience tells me that more than one person will have problems because they forgot to delete a `>` and/or `+` from code when they copy paste the code into their own R sessions. Avoid being that person!

On the top right corner tab, where objects available in the `Environment` are listed, given this is a fresh R session, you should have no objects listed (there might be objects if you used R before and there is already an `.Rdata` file in the working folder). It will look like something as



First we just create a couple of objects and use them, but below we will do it again in more detail. Now we just want to create some objects so that we can then save them and retrieve them again.

```
# assign the value 3 to the object hh2
hh2<-3
# assign the value 5 to the object hh3
hh3<-5
# multiply them up
hh2*hh3
```

```
[1] 15
```

```
# add them up
hh2+hh3
```

```
[1] 8
```

```
#note how you can write comments in R by using "#"
#anything in front of # is not interpreted by R
#and treated as a comment
#you should have the good habit of extensively commenting
#all your code so that you know what you've done
#when you return to it even months or years later
```

We can print an object to the screen by simply typing its name and press enter (despite the fact that currently you can actually see the values on these objects `Environment` tab - but that is because they are simple objects and the workspace is almost empty.).

```
hh2
```

```
[1] 3
```

```
#same as
print(hh2)
```

```
[1] 3
```

Tip: There is actually a simpler way to execute code from the script file in RStudio. `CTRL-Enter` is a keyboard shortcut for "source the current line of code in my script file and move the cursor to the next line". In general if you like keyboard shortcuts, look in RStudio under the menu `Help | Keyboard shortcuts` - there are probably many more than those you will be able to remember!

R is a very powerful calculator! Try some simple maths, say for example (you need to press enter after each line so that the line is evaluated)

```
# R can add!
4+3
```

```
[1] 7
```

```
# and calculate a logarithm (here, of 8)
log(8)
```

```
[1] 2.079442
```

```
#calculates the sin of a number (here of 3.1415)
sin(pi)
```

```
[1] 1.224606e-16
```

```
# or make any kind of calculation really
1234*sqrt(234)-12/23*4^(0.12-0.4)
```

```
[1] 18876.22
```

R will do much more than that, of course. But it can be hard to get going. There will be hundreds or thousands of functions to choose from. The `print`, `log` and `sin` above were just three examples. If you want to know how to use a function, RStudio provides a very useful auto-completion code capability. Try to write this on the command line

```
log()
```

and then put the cursor inside the function parenthesis and press the `Tab` key. RStudio will show you what are the arguments that the function can take (in this case, it's just the number you want the logarithm for, `x`, and `base`, the base of the respective logarithm). Remember this when you are using functions and unsure of what the corresponding arguments are.

It is now time to end our first R session. At this point you need to decide what to do, as all objects created so far are in the memory, but this will be wiped out unless we explicitly save it to a file. The easiest way to do so is by calling the `save.image` function

```
save.image(file="my1stR.Rdata")
```

Note the unusual extension name `.Rdata` associated with R workspaces (an R file is called a workspace). We could now load up this workspace in a new R session, or typically we will load up that workspace by starting R by double clicking on the file created. Do this to see that you retrieve the above created objects. Note that if you already have an R session open, you can load up any previously saved workspace via function `load`.

Finally, just to avoid clutter later, we will delete all the objects created so far

```
#deletes all objects in the dynamic report temporary memory
rm(list = ls())
```

Check nothing remains in your environment

```
ls()
```

```
character(0)
```

If you wanted to work again with whatever you might have saved in your workspace, you would do so by loading the saved `.Rdata` workspace file

```
load(file="my1stR.Rdata")
```

as you can see, the objects you had created are back and available to you

```
ls()
```

```
[1] "hh2" "hh3"
```

We will not be using these objects anymore, so feel free to delete them as well as to delete the workspace file `my1stR.Rdata`.

Note that you saved your workspace in some directory, and loaded it up again from there, but you have not defined the directory explicitly. By default, this is your working directory. You can check what that directory currently is by using the following command

```
getwd()
```

```
[1] "C:/Users/tiago/Documents/GithubDesktop/AnIntro2RTutorial"
```

You can always change the directory you are working on by setting it up explicitly to your desired location, using

```
#set the working directory - but remember to use your own path!!!
setwd("C:/Users/tiago/Desktop/mycourse")
```

It is a very good habit to make sure that you are working in the directory you think you are working. Many errors might occur if R can't find some object or file because it is looking on the wrong place.

A good trick to make sure you are working in the directory you want is to open RStudio from the directory you want it to be working on. You can do that by double clicking a file R studio recognizes as a file it should open, e.g. as might happen in particular for files with extensions `.R` (a script), a `.Rdata` (a workspace) or a `.qmd` (a dynamic report in Quarto).

If you are already in RStudio, you can also use the `Files` tab to see where you are, move to the folder where you want to be working, and then from the `Files|More` menu (a dented wheel) select the option `Set as working directory`.

Now you have used R in RStudio, lets use the power of their integration to work directly in a dynamic report.

Working through R via a dynamic report

Create a new dynamic report using a template quarto file. To make it easier, use the template provided here:

https://github.com/TiagoAMarques/template_4_quarto_report

put all the files in a folder, rename the template file `QuartoTemplate.qmd` to something more sensible, something like `*****XX2026tutorial.qmd`. Here `*****` could be your FCUL student number and `XX` a course identifier (e.g. EN for Ecologia Numérica or ME for Modelação Ecológica).

Then start your work in a new section at the top. ater you can read the text in the tempate to find out more about how different things get added to the output. This template allows you to obtain `.pdf`, `.docx` and `.html` as outputs.

Comment all you do in the appropriate place: things that seem obvious to you when you do them won't necessarily be obvious to you next week, so describe with comments what they are and why you did them. Therefore, at the end you will have a record that makes it easy to track everything you did, and a template you can use in future classes.

Once you created the quarto file from scratch, we can start by creating a new variable.

Note that all the code must go inside code chunks, and you can get them by doing `Code | InsertChunk` or the shortcut `Ctrl+Alt+I`. When code chucks are properly formatted they appear highlighted in the .Rmd.

An empty code chunk (in the image with a comment added to it!) looks like this:



We will create a variable called `myvar1` which we will assign the value of 4. This is typically done using the assign operator `<-`.

```
myvar1 <- 4
```

There are typically multiple ways to do the same thing in R, and this is sometimes referred to as a disadvantage by lets call them "code-purists". For simplicity, we deliberately avoid presenting the several alternatives for each action, and concentrate on the ones we prefer. This is not the same as saying these are the best, and if you continue to work with R you will likely get used to doing things your way - for now we do it our way!

An object should have been created in your workspace. You can list all objects in a given workspace using

```
ls()
```

```
[1] "hh2"    "hh3"    "myvar1"
```

You can also remove any object by using the`rm` function, so here we remove `myvar1`.

```
rm(myvar1)
```

and hence our workspace is empty again.

Task 0: Create some objects and assign numbers to them. Then try to make some basic calculations with the objects you just created. Finally, clean up the workspace again.

Note a key difference between the functions `ls` and `rm`. While the first function does not need any arguments, the second requires at least one argument (but can take several). This can be easily seen by checking their help files and noting that `rm` needs at least 1 explicit argument while `ls` can work with defaults

```
# do not have this line of code in a dynamic report
# it will fire the help file into a browser
# which is a bit of a pain
# Note in my .Rmd this code has
# eval=FALSE and so is not executed, just shown
?rm
```

This is a convenient way to obtain more information about a given function. If one does not know what the name of the function might be, one can search for functions containing a given string using the `apropos` function. The following command lists all the functions with the string `mean` in them.

```
apropos("mean")
```

```
 [1] ".colMeans"     ".rowMeans"     "colMeans"      "kmeans"
 [5] "mean"          "mean.Date"     "mean.default"  "mean.difftime"
 [9] "mean.POSIXct"  "mean.POSIXlt"  "rowMeans"      "weighted.mean"
```

Not surprisingly, most if not all of these functions will be used for some kind of calculation involving a mean. You can look into any one of them using the `?` as above. We have assigned a number to a variable , but we can actually more generally have vectors (strictly, `myvar1` was a numeric vector of length 1) containing a large number of values "inside" them.

The following code assigns some numbers to 5 different vectors.

```
x2<-c(1,2,0.12,4,-22)
x3<-seq(1,8,by=2)
# : useful shortcut for sequences with the by argument = 1
x1<-1:5
z1<-10:8
z2<--10:10
```

Take a peak at the objects just created (important: you must type an object name to see it printed on the output!):

```
x1
```

```
[1] 1 2 3 4 5
```

```
x2
```

```
[1]   1.00   2.00   0.12   4.00 -22.00
```

```
x3
```

```
[1] 1 3 5 7
```

```
z1
```

```
[1] 10  9  8
```

```
z2
```

```
 [1] -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4   5   6   7   8
[20]   9  10
```

The function `seq` is very useful for setting sequences of numbers. The optional arguments `by`, `length.out` and `along.with` provide extra flexibility. Look at `?seq` to find out what the function does and the consequences of using these different arguments. In our experience `by` and `length.out` (or `length` for short) will be more often used than `along.with` (or `along` for short), but this you will find out for yourself from experience.

We can use the usual mathematical operators over vectors. A few examples follow:

```
x1+x2
```

```
[1]   2.00   4.00   3.12   8.00 -17.00
```

```
x4<-x1+x2
x5<-x1-x2
x6<-x1*x2
x7<-x1/x2
```

Note by default you do not see results, you need to print them to the report to see them. As an example

```
print(x4)
```

```
[1]   2.00   4.00   3.12   8.00 -17.00
```

if you just use the name of the object on the console it also gets printed by default

```
x4
```

```
[1]   2.00   4.00   3.12   8.00 -17.00
```

Note that if the vectors are of the same length, R performs the operation element-wise. Another useful (but possibly dangerous) feature is that R *recycles* vectors if they are not the same length

```
x8<-c(1,2,3,4)
x8+2
```

```
[1] 3 4 5 6
```

However, if one of the vectors is smaller, unexpected behavior can happen, because R recycles elements regardless (so be careful, a warning is typically produced)

```
x9<-c(3,4,5)
x10<-c(0.7,0.9,1.3)
x9+x10
```

```
[1] 3.7 4.9 6.3
```

```
x8+x9
```

```
Warning in x8 + x9: longer object length is not a multiple of shorter object
length
```

```
[1] 4 6 8 7
```

As expected, a warning message was produced when `x8` and `x9` were added. Usually *error messages are important and should be read*! Quite often the answer to your current question lies in the previous error or warning message.

Another useful function is `rep`, which allows one to create repetitions of patterns. As examples, see the difference between the next two lines of code

```
rep(c(1,2,3,4),times=3)
```

```
 [1] 1 2 3 4 1 2 3 4 1 2 3 4
```

```
rep(c(1,2,3,4),each=3)
```

```
 [1] 1 1 1 2 2 2 3 3 3 4 4 4
```

We have just created and removed some objects, and used simple functions like `ls`, `seq` or `save`. R is an object oriented language, and functions and vectors are just examples of types of objects available in R. In the next section we go through the most commonly used classes of objects in R.

Types and classes of objects

Objects can have classes, which allow functions to interact with them. Objects can be of several classes. We already used the class `numeric`, which is used for general numbers, but there are also additional very commonly used classes:

- `integer`, for integer numbers (could save memory with respect to the class numeric, if you only have integers)
- `character`, representing for character strings, like "words" say
- `factor`, used to represent levels of a categorical variable, e.g. car makes or models
- `logical`, the logical values TRUE and FALSE
- `matrix`, an object containing data in tabular form

- `data.frame`, an object containing the kind of data you could have in a single Excel worksheet
- `list`, an object that can be used to store all sorts of unstructured (or structured) data
- `function`, all the functions in R, like those you just used `c`, `seq` or `rep` are of this class.

While many, maaaaany other classes of objects exist, these are arguably some of those more commonly used.

Outputs of some analyses have special classes, as an example, the output of a call of function `lm` is an object of class `lm`, i.e., a linear model. Many packages introduce special classes for objects, so that functions know how to behave when those objects are used as arguments. Typically, functions behave differently according to the class of the objects that are used as their arguments. As an example, note below how `summary` treats differently an object of class `factor` or one of class `numeric`, producing a table of counts per level for a factor but a 6 number summary for numeric values.

```
obj1<-factor(c("a","a","b","a","b","a","a","a","a","a","e","a","f","a","f","d","a","b","d","a
summary(obj1)
```

```
 a  b  d  e  f
14  4  3  1  2
```

```
obj2<-c(2,5,-0.2,89,12,-3,-5.4)
summary(obj2)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   -5.4    -1.6     2.0    14.2     8.5    89.0
```

We can check the class of an object using function `class`, as in the following examples

```
class(obj1)
```

```
[1] "factor"
```

```
class(obj2)
```

```
[1] "numeric"
```

```
class(TRUE)
```

```
[1] "logical"
```

It is sometimes useful to coerce, i.e. force, objects into different classes, but care should be used when doing so. Some examples are presented below. Can you describe in your own words what R did below?

```
as.integer(c(3,-0.3,0.4,0.6,0.9,13.2,12))
```

```
[1]  3  0  0  0  0 13 12
```

```
as.numeric(c(TRUE,FALSE,TRUE))
```

```
[1] 1 0 1
```

```
as.numeric(obj1)
```

```
 [1] 1 1 2 1 2 1 1 1 1 1 4 1 5 1 5 3 1 2 3 1 2 1 1 3
```

A common way to organize multiple vectors together is in the form of a matrix. Here we create such an object

```
mat1<-matrix(1:12,nrow=3,ncol=4)
mat1
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

Note that by default R fills the first column (with 1,2,3) then the second column (4,5,6) etc. If you want it to fill the matrix by rows, i.e. the first row, then the second, etc, you can use the optional argument `byrow=TRUE`, like this:

```
matrix(1:12,nrow=3,ncol=4,byrow=TRUE)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
```

R also allows data structures with more than 2 dimensions – we don't cover those here, but look up the help on `array` if you are interested. A matrix is just a two dimensional array.

Arrays are useful objects, but can be complex to visualize due to their potential high dimensionality. Another common type of object is a `data.frame`. This is essentially a matrix but for which each column can be of a different type. These are what we would typically associate with an excel spreadsheet or a table in a database. Typically columns correspond to variables observed in a number of subjects, each subject recorded in its own row. A simple example with 3 variables and 5 subjects follows:

```
mysex<-c("male","female","female","male","male")
myage<-c(34,23,56,45,12)
myhei<-c(185,178,167,165,148)
df1<-data.frame(ID=1:5,sex=mysex,age=myage,height=myhei)
df1
```

```
  ID    sex age height
1  1   male  34    185
2  2 female  23    178
3  3 female  56    167
4  4   male  45    165
5  5   male  12    148
```

Typically, `data.frames` are used to store the data we subsequently analyse. Usually the data are not manually imputed as above, but read into R from other software, using R functions addressed in a later section.

A data frame is just a special type of `list`. A `list` can contain objects of different types and dimensions. An example is here

```
list1<-list(Note="whatever I want here",X2=4,age=1:4)
list1
```

```
$Note
[1] "whatever I want here"

$X2
```

```
[1] 4

$age
[1] 1 2 3 4
```

Lists are typically used to store outputs of computations which require different kinds of objects to be recorded. Note the use of `$` to access the sub components of a list or a data.frame.

```
list1$X2+10
```

```
[1] 14
```

Alternatively, one might use index to retrieve elements of a list

```
list1[[3]]+5
```

```
[1] 6 7 8 9
```

In the next section we will learn more about using indexes to access subsets of data.

Subsetting data

One useful feature of R relates to how we can index subsets of data. The indexing information is included within square brackets:`[ ]`. As an example, we can select the 3rd element of a vector

```
x<-c(1,3.5,7,8,-7,0.43,-1)
x[3]
```

```
[1] 7
```

but we can also select all except the second and third elements of the same vector

```
x[-c(2,3)]
```

```
[1]  1.00  8.00 -7.00  0.43 -1.00
```

We can also select only the objects which follow a given condition, say only those that are positive

```
x[x>0]
```

```
[1] 1.00 3.50 7.00 8.00 0.43
```

or those between (-1,1)

```
x[(x>-1) & (x<1)]
```

```
[1] 0.43
```

Note the subtle difference between the previous and next statements

```
x[(x>=-1) & (x<=1)]
```

```
[1]  1.00  0.43 -1.00
```

which reminds us we should be careful when setting these logical conditions, especially when working with integer boundaries which might be on the limits of those conditions. Note indexing can be done using additional information. As an example, we select here the elements in `x` such that the corresponding elements in `y` are positive:

```
#rnorm(k) produces k Gaussian random deviates
x<-rnorm(10)
y<-rnorm(10)
x2<-x[y>0]
```

When working on a matrix the indexing is done by row and column, therefore for selecting the value that is in the third row and second column of a matrix we use

```
mat1[3,2]
```

```
[1] 6
```

but we can also select all the elements in the second row

```
mat1[2,]
```

```
[1]  2  5  8 11
```

or the fourth column

```
mat1[,4]
```

```
[1] 10 11 12
```

We are often interested in subsetting a dataset by some characteristic of one (or several) of its columns. Here we illustrate with the dataset `iris` (check `?iris` for data details)

```
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

```
str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

that contains information about 3 species: setosa, versicolor and virginica. Imagine that we want to do something just with those from species virginica. Then we can create an object holding just that information as

```
iris.3 <- iris[iris$Species=="virginica",]
summary(iris.3)
```

```
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.900   Min.   :2.200   Min.   :4.500   Min.   :1.400
 1st Qu.:6.225   1st Qu.:2.800   1st Qu.:5.100   1st Qu.:1.800
 Median :6.500   Median :3.000   Median :5.550   Median :2.000
 Mean   :6.588   Mean   :2.974   Mean   :5.552   Mean   :2.026
 3rd Qu.:6.900   3rd Qu.:3.175   3rd Qu.:5.875   3rd Qu.:2.300
```

```
 Max.   :7.900   Max.   :3.800   Max.   :6.900   Max.   :2.500
       Species
 setosa    : 0
 versicolor: 0
 virginica :50
```

Mathematical functions

Within R there are a number of mathematical operators but also mathematical and statistical functions. As any other functions, many of these have required parameters and optional parameters. It would take a very long time to describe even the most basic functions. Therefore, we prefer to let you try hands on explore a number of these.

Task 1: Take your time to explore the functions below: `sum(x)`, `sqrt(x)`, `log(x)`, `log(x,n)`, `exp(x)`, `choose(n,x)`, `factorial(x)`, `floor(x)`, `ceiling(x)`, `round(x,digits)`, `abs(x)`, `cos(x)`, `sin(x)`, `tan(x)`, `acos(x)`, `acosh(x)` , `max(x)`, `min(x)`, `mean(x)`, `median(x)`, `range(x)`, `var(x)`, `cor(x,y)`, `quantile(x)`.

(Tip: do not forget that you can get a full description what each function can be used for, what arguments it takes, and what kind of output it produces, using `?`. Further, the help of most functions includes examples of their use, which proves invaluable to understand their usage.)

Extending basic capabilities via packages

While R base installation includes enough functions that getting acquainted with them could take several years, many more are available via the installation of additional packages available online on CRAN (The Comprehensive R Archive Network).

A package is just a set of functions and/or data sets (and the corresponding documentation plus some additional required files) which usually have some specific goal. As examples, in our course we will be using general packages `lubridate`, great to manipulate dates and times, `dplyr`, which extends data import and manipulation, as well as `vegan` and `mgcv`, which allow the implementation of a variety of numerical ecology techniques and generalized additive models (GAM), respectively.

Note packages cover a very wide range of applications, and chances are that at least a package, often more than one, already exists to implement most kinds of statistical or data processing tasks we might imagine. Therefore, before coding your own new method, check if it has been coded by others before.

Installing a new package in R requires a call to function `install.packages()`. A handy RStudio shortcut is simply to follow the `Tools|Install packages...` shortcut.

After a package is installed it needs to be loaded to be available. In R this is done calling function `library()` with the package name as an argument. In RStudio this can also be done by ticking the corresponding package box under the RStudio tab "Packages" (by default this tab is available on the bottom right window, between the "Plots" and "Help" tabs).

Here we use package `praise` as an example. You might be needing it by now! Notice to begin with that `praise` is not available yet (assuming you never installed it in the machine you are working on)

```
# do not have this line of code in a dynamic report
# it will fire the help file into a browser
# which is a bit of a pain
# Note in my .Rmd this code has
# eval=FALSE and so is not executed, just shown
?praise
```

Next, we install the package.

```
# do not have this line of code in your dynamic report
# the function "install.packages"
# will install the package praise
# and that only needs to be done once
# otherwise it would be like
# installing word every time
# you wanted to work on a word file
# in the .Rmd the option is set as
# eval=FALSE and so is not executed, just shown
install.packages("praise")
# if you need to install praise for the first time
# copy paste the above line in the command line
# and then press enter - done
```

Then we load the package

```
library("praise")
```

and finally we check that the `praise` functions are now loaded

```
# praise() is actually the only function
# meant to be used
# by users in the praise package
# do not have this line of code in a dynamic report
# it will fire the help file into a browser
# which is a bit of a pain
# Note in my .Rmd this code has
# eval=FALSE and so is not executed, just shown
?praise
```

From now on, when you feel frustrated about R (or life in general…) you can just use R to uplift you

```
# get uplifted
praise()
```

```
[1] "You are slick!"
```

Important note: you can and should load required packages in dynamic reports via the function `library`. You should not install packages (with `install.packages()`) within a dynamic report. Install packages directly on the command line, and only once.

Otherwise, if doing it inside a dynamic report, that would be a akin to installing Word every time you wanted to work on a Word document!

Task 2:

1. Use the internet to find some useful package to do something you might want to do, the install the package and then use the example code in it.
2. Investigate the package `memer` to create memes from within R :)

An example of the required code is below

```
# if devtools is not installed, 1st install it
# install.packages("devtools")
# some packages are not available from CRAN
# being available at their developers page
# Many of these are hosted on github
# Install the development version of memer from GitHub:
# This might require you to install dependencies
# (dependencis: other packages that a package needs)
# and all that could take some time
```

```
# so do not do this in class
devtools::install_github("sctyner/memer")
# load the package
library(memer)
# Create a meme
meme_get("DistractedBf") %>%
  meme_text_distbf("tidyverse", "new R users", "base R")
# how cool is that
# challenge: send me a new meme that you create yourself
```

Both `memer` and `praise` are examples of "fun" packages. There are a few more you can explore here.

So, in fact, unlike what it might have been your first impression, R can be fun :)

Importing and exporting data

Rather than importing data into R manually, typically the data we work with are imported from some external source. Typically this might be some simple file format, like a txt or a csv file, but while not covered here, direct import from say Excel files or Access data bases is possible. Such more specialized inputs often require additional packages.

RStudio includes a useful dedicated shortcut `Import dataset`, by default available through the top right window of RStudio's interface. Note this shortcut essentially just calls the appropriate functions required for each import. Here we present a couple of examples just for practicing.

First, we load up a data frame which exists in R (note R includes a large variety of example data sets which are useful to illustrate the use of code) and contains an example data set, with variables measured in 150 flowers of 3 varieties. This is in object `iris`, and we use the function `data` to load it so that we have access to it.

```
data(iris)
```

we can take a look at what this data set contains

```
#example of head use: see the first 4 rows in iris
head(iris,4)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
```

```
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
```

```
#example of str use
str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
#example of summary use
summary(iris)
```

```
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
 Median :5.800   Median :3.000   Median :4.350   Median :1.300
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
       Species
 setosa    :50
 versicolor:50
 virginica :50
```

Now we create a new data frame which we then modify to include a new variable

```
mydata<-iris
mydata$total<-mydata$Sepal.Length+mydata$Sepal.Width+
mydata$Petal.Length+mydata$Petal.Width
```

Now, we are going to export this data set as a txt, named `mydatafile.txt`

```
write.table(mydata,file="mydatafile.txt",row.names=FALSE)
```

Note the use of the optional argument `row.names=FALSE`, otherwise some arbitrary row names would be added to the file. If you look in the folder you are working in, you should now have a new file there. Open it and check that it looks as you would expect. Next, we are going to import it back into R, into an object named `indat`.

```
indat<-read.table(file="mydatafile.txt",header=TRUE)
```

So now we have our data back in R.

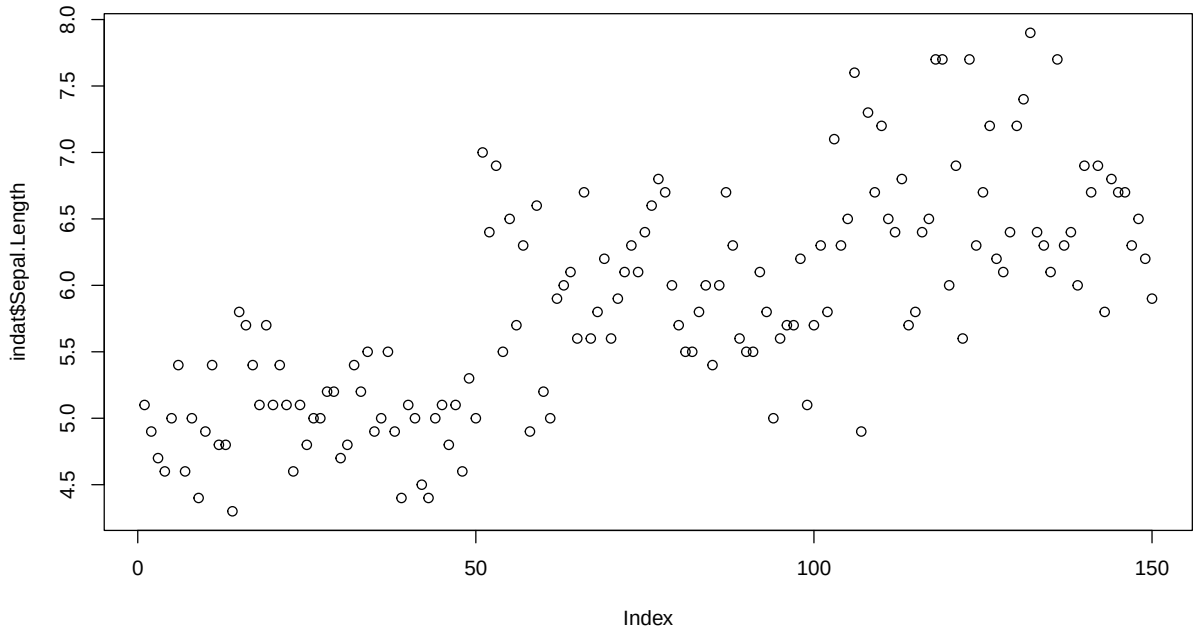
Task 3: Import the file `dados1.csv` into R, giving it the name `newfile`. Tips: Explore the possible options including (1) `Import Dataset` shortcut in the `Environment tab`, which is usually a convenient way to find the right function with suitable defaults for your data (2) the optional arguments in function `read.table` above or (3) consider using function `read.csv`.

Plotting data

Base R

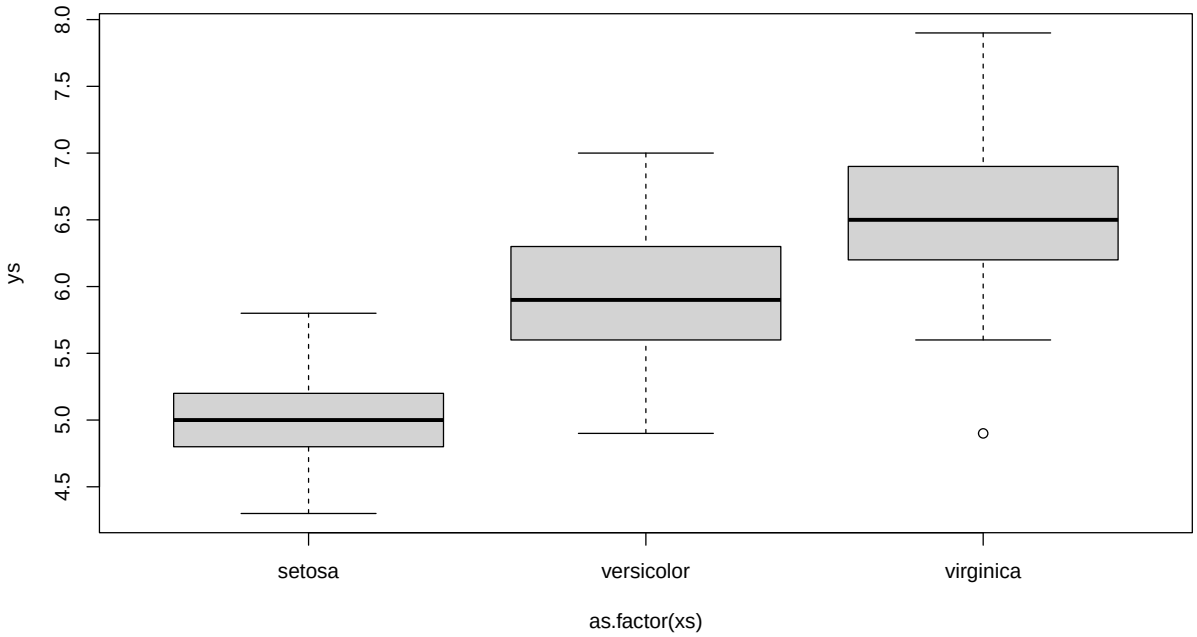
One of the most amazing R capabilities are its graphics customization properties. One can create pretty much any graphic output desirable. The plot function is, as we have seen before for function `summary`, a function that attempts to do something smart depending on the type of arguments used. Using the data set iris previously considered, plot examples are implemented below, with some optional arguments being used to show some of the possibilities to customize plots.

```
#default use
plot(indat$Sepal.Length)
```



In the following example, R evaluates the class of one of the arguments as being a factor and hence tries to give you a sensible result, which is producing a boxplot of a numerical variable as a function of a factor.

```
ys<-indat$Sepal.Length
xs<-indat$Species
#note use of ~ to represent "as a function of"
plot(ys~as.factor(xs))
```



Note the use of ~ to mean "as a function of"; this is also used below when specifying regression models, where the object on the left of ~ will be the response variable and the objects on the right explanatory variables.

We now add some labels to a new plot, using directly function `boxplot` (which in the background `plot` above called), of sepal length as a function of species

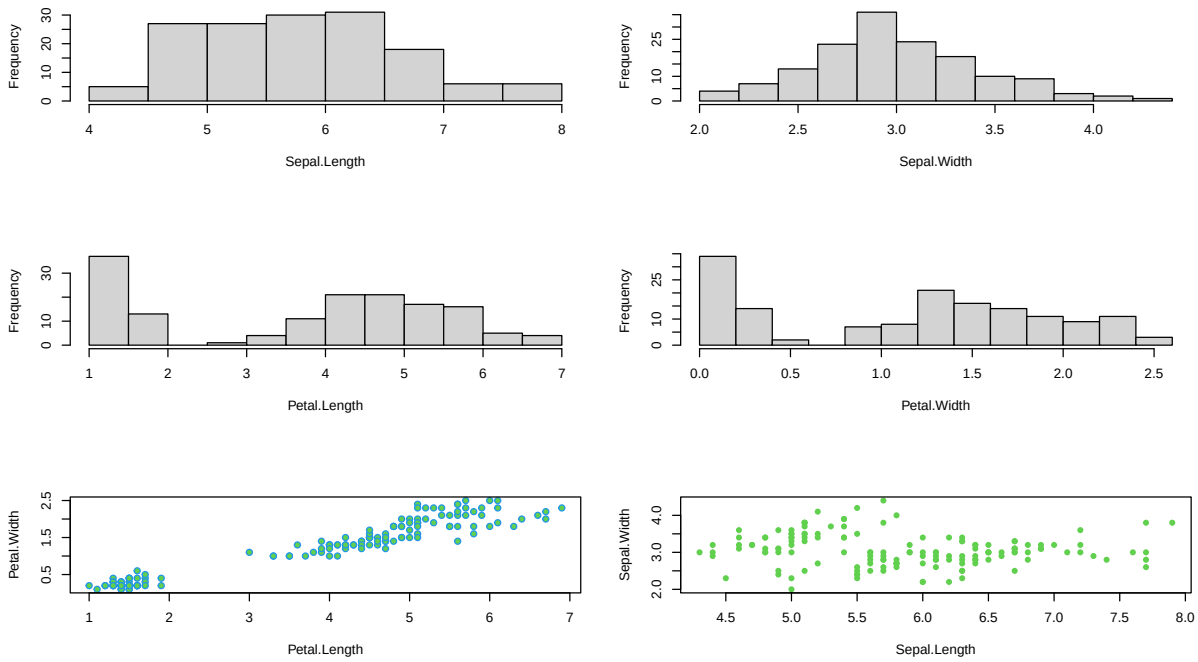
```
ys<-indat$Sepal.Length
xs<-indat$Species
#note use of ~ to represent "as a function of"
boxplot(ys~xs,ylab="Sepal Length (in mm)",main="Sepal length by species")
```




```
#compare with this code - next line returns an error
#plot(ys~xs,ylab="Sepal Length (in mm)",main="Sepal length by species")
#making species be a factor - allows the plot below to work well
#xs<-as.factor(indat$Species)
#plot(ys~xs,ylab="Sepal Length (in mm)",main="Sepal length by species")
```

We can also set the graphic window to hold multiple plots. This is obtained via argument **mfrow**, one of the arguments in function. Note this function controls a much larger number of graphical parameters. You can take a look at its help file to get a feel for how many and what kind of control it allows you. An example follows, in which we leverage on the use of function **with** to avoid having to constantly use **indat$** to tell R where the data can be found.

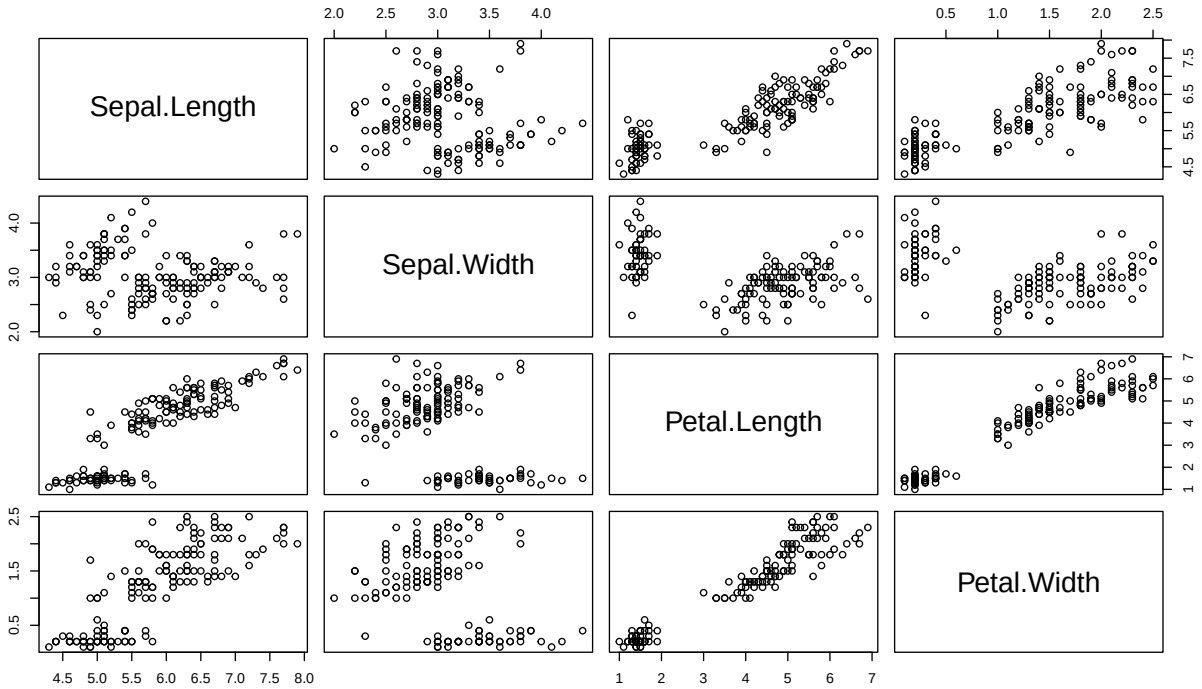
```
#define 3 rows and 2 columns of plots
par(mfrow=c(3,2))
with(indat,hist(Sepal.Length,main=""))
with(indat,hist(Sepal.Width,main=""))
with(indat,hist(Petal.Length,main=""))
with(indat,hist(Petal.Width,main=""))
with(indat,plot(Petal.Length,Petal.Width,pch=21,col=12,bg=3))
with(indat,plot(Sepal.Length,Sepal.Width,pch=16,col=3))
```



We used argument `mfrow`, but looking at the help for function `par` gives you an insight to the level of customization one can reach with respect to these graphical parameters, via dozens of different arguments.

We can look at the correlation structure between all variables using function `pairs`.

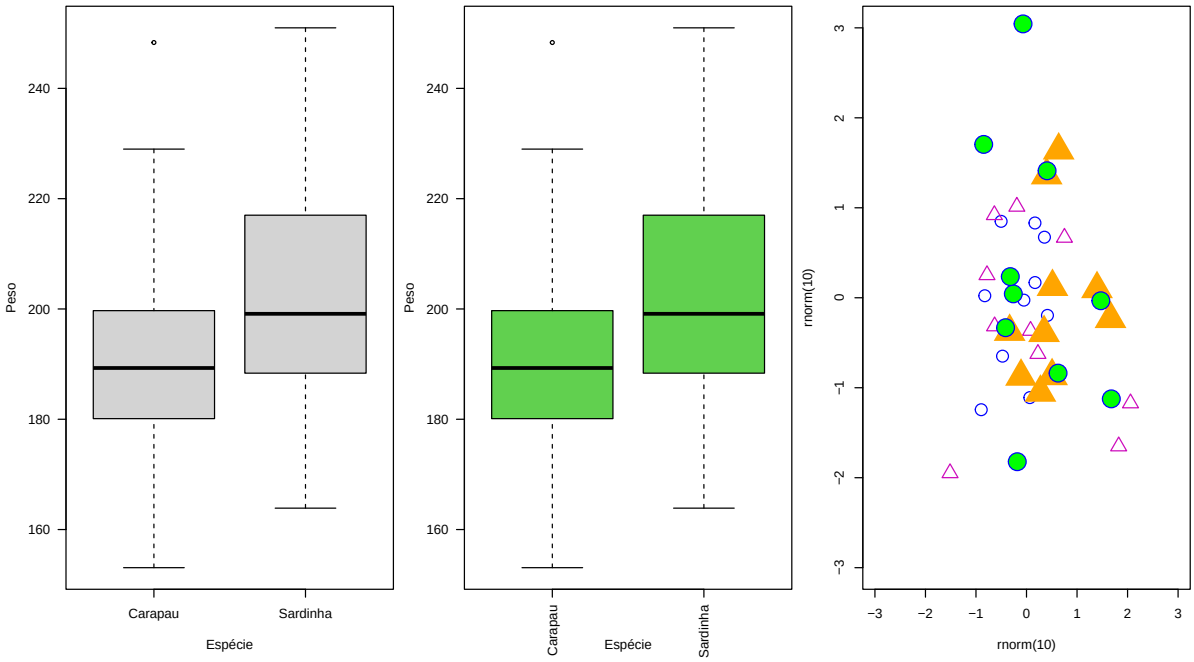
```
#note selection of just the first 4 columns, since the last is not numeric
pairs(indat[,1:4])
```



Task 4: Using data `cars`, create a plot that represents the stopping distances as a function of the speed of cars. Use the `points` function to add a special symbol to points corresponding to cars with speed lower than 15 mph, but distance larger than 70m. Check out the function `text` to add text annotations to plots. Customize axis labels.

Note that we can control most, if not all, elements of a plot. As an example, see the following code, where I am controlling all sorts of aspects. Search the help file to understand what the arguments `mar` of function `par` does (sets margin sizes) as well as `plot`'s parameters `xlim`, `ylim` `cex`, `pch`, `col`, do. See `?par` to check all the graphical parameters you can control on plots.

```
set.seed(1234)
par(mfrow=c(1,3),mar=c(8,4,0.5,0.5))
pesos<-rnorm(100,200,20)
sps<-rep(c("Carapau","Sardinha"),each=50)
boxplot(pesos~sps,xlab="Espécie",ylab="Peso",las=1)
boxplot(pesos~sps,xlab="Espécie",ylab="Peso",las=2,col=3)
plot(rnorm(10),rnorm(10),xlim=c(-3,3),ylim=c(-3,3),col="blue",cex=2)
points(rnorm(10),rnorm(10),col=6,cex=2,pch=2)
points(rnorm(10),rnorm(10),col="orange",cex=4,pch=17)
points(rnorm(10),rnorm(10),col="blue",cex=3,pch=21,bg="green")
```



Here and in class I will generally use R base for plots, but nowdays the cool kids tend to use the ggplot2 system (https://r-graph-gallery.com/ggplot2-package.html). It is worth to go and find out about it, but also see next section for a sneack preview.

If you really want to get creative with your plots, and a great plot can be extremely useful in conveying a message, check out the gallery here: https://r-graph-gallery.com/ Lots of ideas, with the R code to use them with your data!

Tidyverse

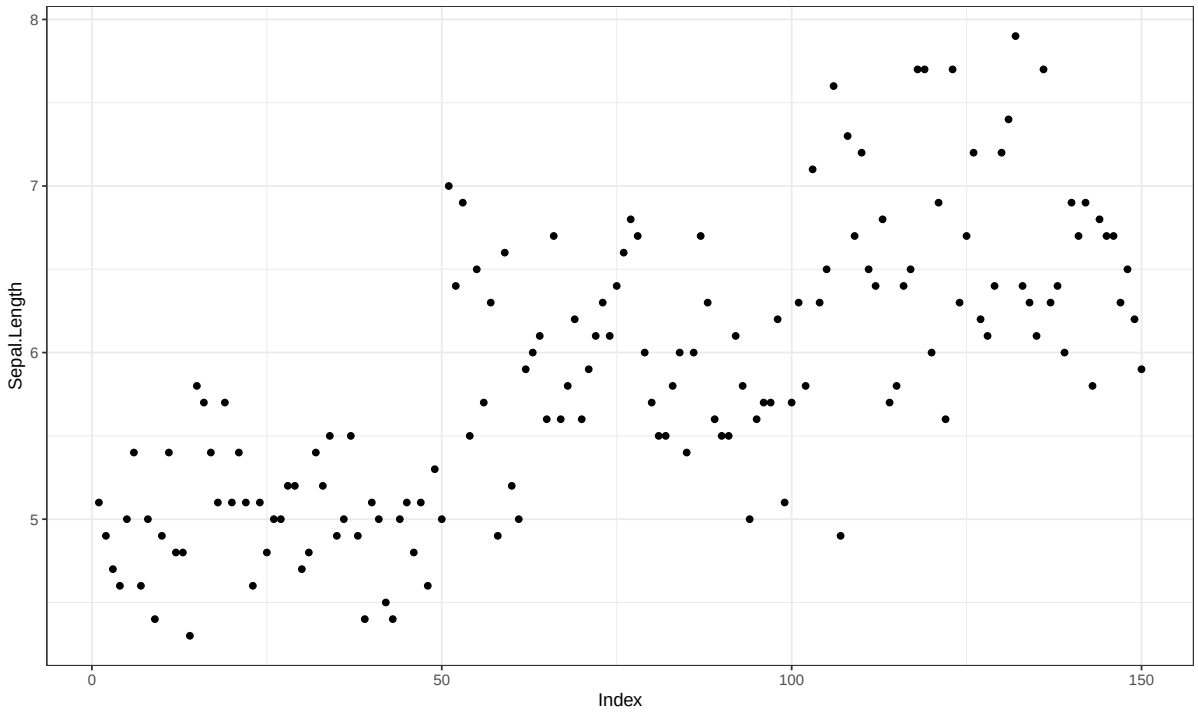
Here I just present the above plots considering the `ggplot2` syntax, so you can get used to seeing it both ways, and also starting to decide which way you prefer to work yourself. First, we load up ggplot2

```
library(ggplot2)
```

Then, we reproduce the five plots created above in base R. Do it yourself, and changes as much as you can. Break it too, learn from mistakes!

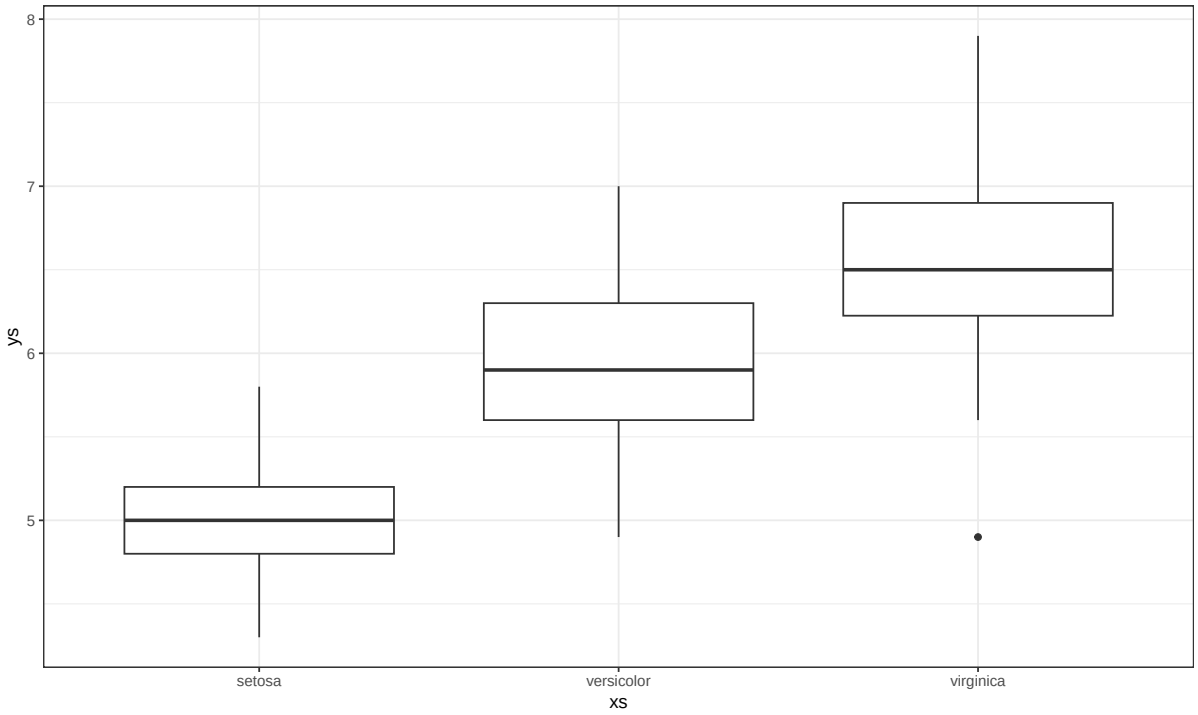
First plot

```
ggplot(indat, aes(x = seq_along(Sepal.Length), y = Sepal.Length)) +
  geom_point() +
  labs(x = "Index", y = "Sepal.Length") +
  theme_bw()
```



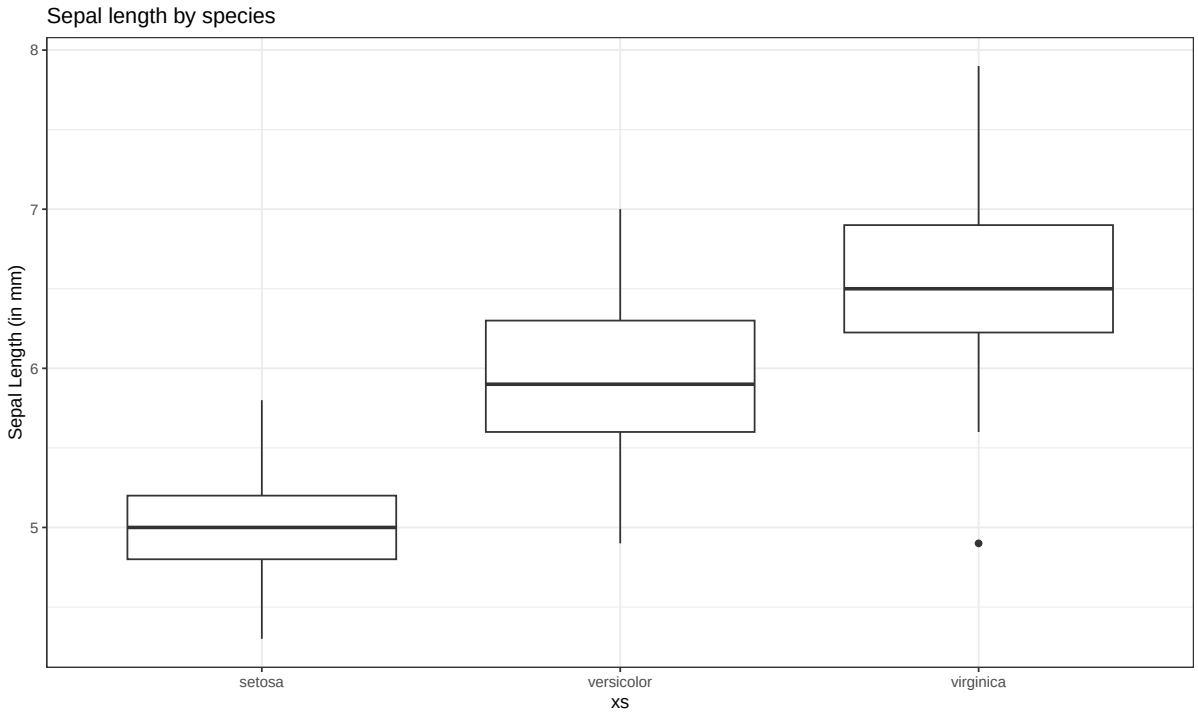
Second plot

```
ys<-indat$Sepal.Length
xs<-indat$Species
ggplot(indat, aes(x = as.factor(xs), y = ys)) +
  geom_boxplot() +
  labs(x = "xs", y = "ys") +
  theme_bw()
```



Third plot

```
ys<-indat$Sepal.Length
xs<-indat$Species
ggplot(indat, aes(x = as.factor(xs), y = ys)) +
  geom_boxplot() +
  labs(x = "xs",
       y = "Sepal Length (in mm)",
       title = "Sepal length by species") +
  theme_bw()
```



Fourth plot

```
library(ggplot2)
library(patchwork)

p1 <- ggplot(indat, aes(x = Sepal.Length)) +
  geom_histogram() +
  labs(x = "Sepal.Length", y = "Frequency") +
  theme_bw()

p2 <- ggplot(indat, aes(x = Sepal.Width)) +
  geom_histogram() +
  labs(x = "Sepal.Width", y = "Frequency") +
  theme_bw()

p3 <- ggplot(indat, aes(x = Petal.Length)) +
  geom_histogram() +
  labs(x = "Petal.Length", y = "Frequency") +
  theme_bw()

p4 <- ggplot(indat, aes(x = Petal.Width)) +
  geom_histogram() +
```

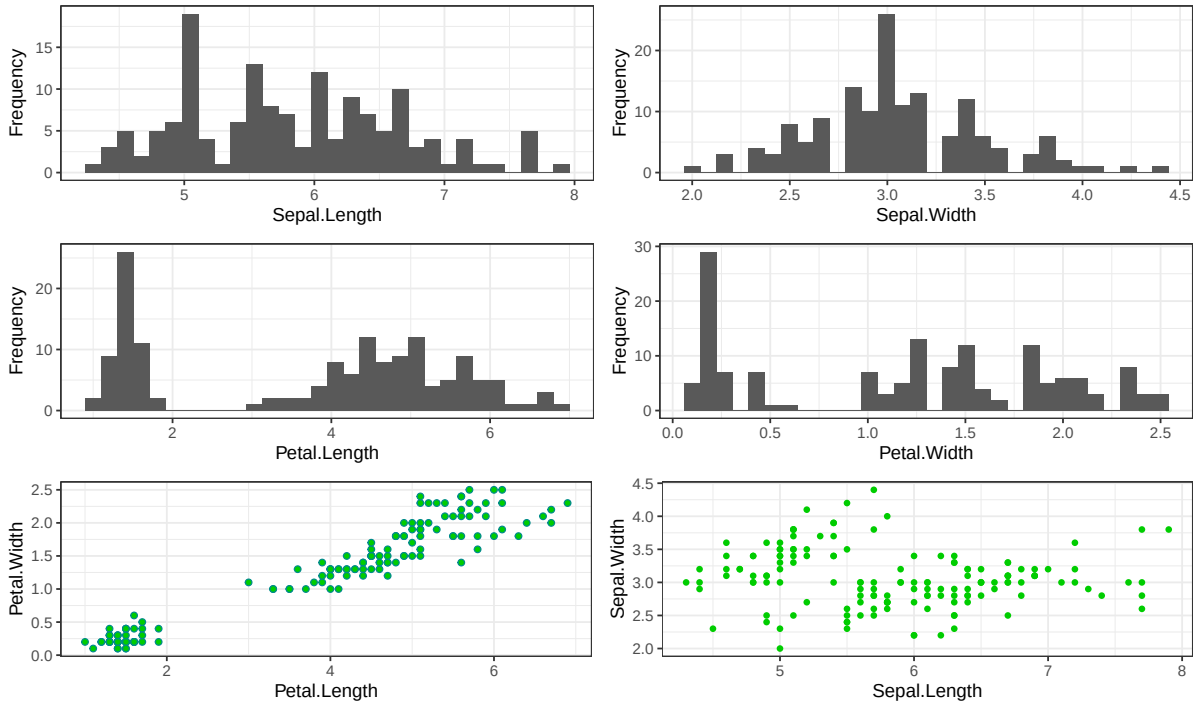
```
  labs(x = "Petal.Width", y = "Frequency") +
  theme_bw()

p5 <- ggplot(indat, aes(x = Petal.Length, y = Petal.Width)) +
  geom_point(shape = 21, colour = "cyan4", fill = "green3") +
  labs(x = "Petal.Length", y = "Petal.Width") +
  theme_bw()

p6 <- ggplot(indat, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(shape = 16, colour = "green3") +
  labs(x = "Sepal.Length", y = "Sepal.Width") +
  theme_bw()

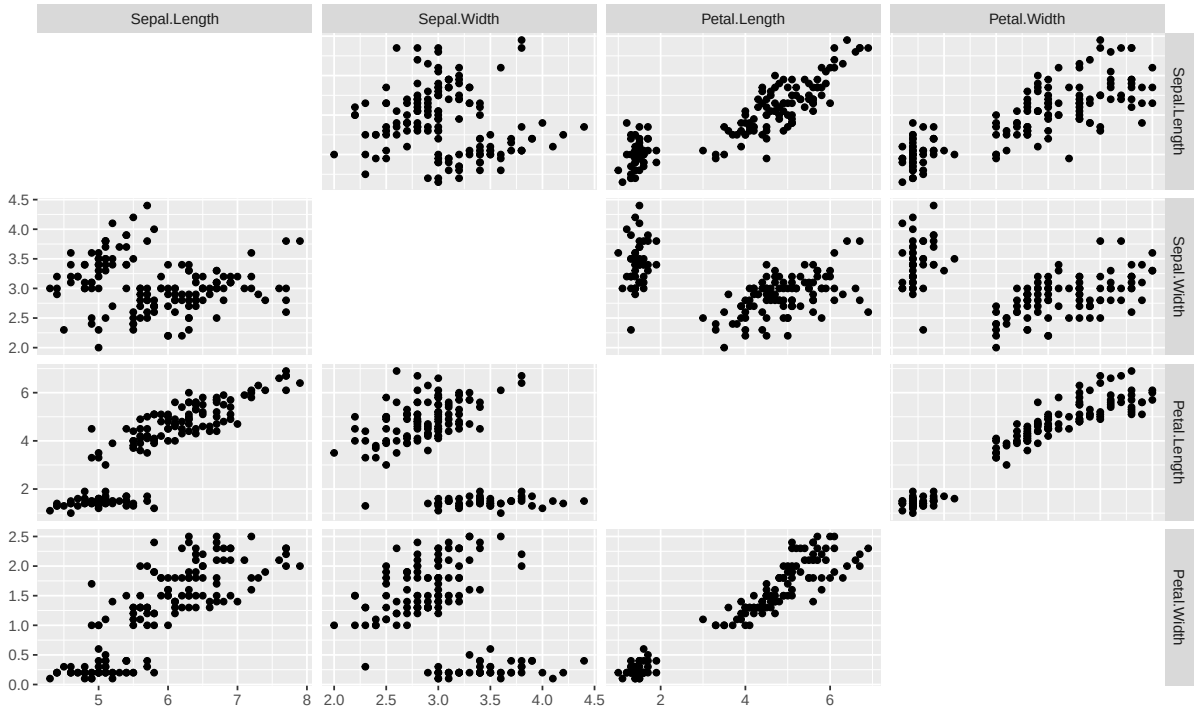
(p1 | p2) / (p3 | p4) / (p5 | p6)
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Fifth plot

```
library(GGally)
ggpairs(indat[, 1:4],
        upper = list(continuous = "points"),
        diag  = list(continuous = "blankDiag"))
```



Sixt plot

```
set.seed(1234)
pesos <- rnorm(100, 200, 20)
sps   <- rep(c("Carapau", "Sardinha"), each = 50)
df    <- data.frame(pesos, sps)

# Boxplot 1 - horizontal x-axis labels (las=1)
p1 <- ggplot(df, aes(x = sps, y = pesos)) +
  geom_boxplot() +
  labs(x = "Espécie", y = "Peso") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5),
        plot.margin = margin(b = 40))

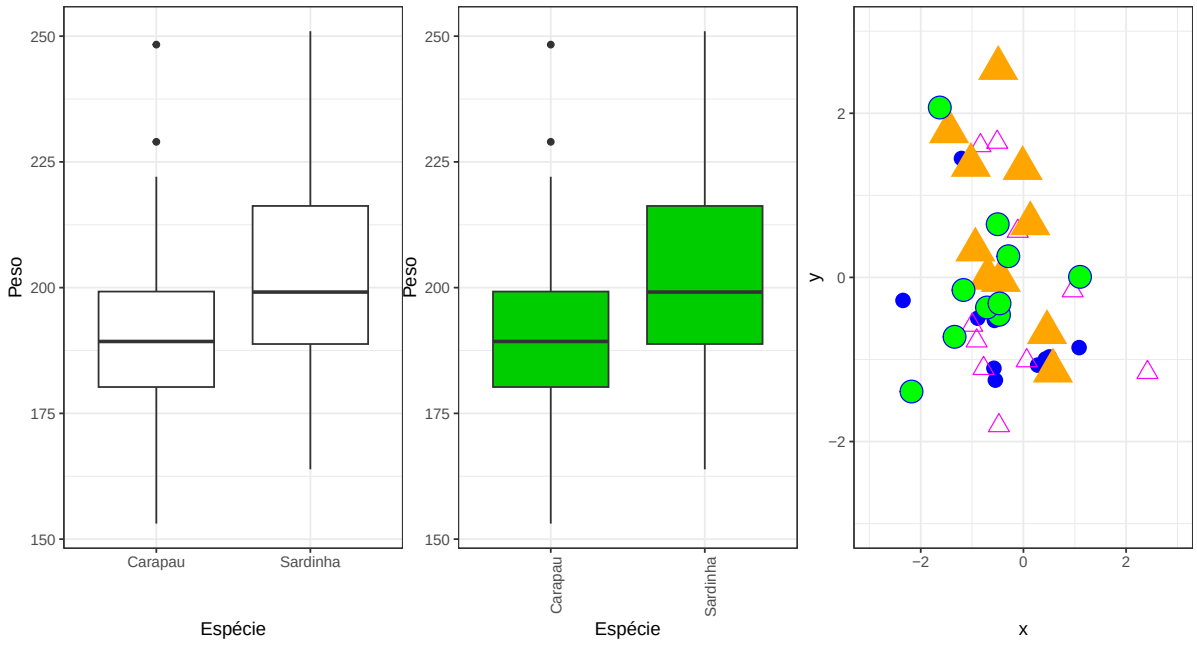
# Boxplot 2 - vertical x-axis labels (las=2) + green fill
p2 <- ggplot(df, aes(x = sps, y = pesos)) +
```

```
  geom_boxplot(fill = "green3") +
  labs(x = "Espécie", y = "Peso") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1),
        plot.margin = margin(b = 40))

# Scatter plot with 4 point layers
set.seed(1234)
df2 <- data.frame(
  x   = c(rnorm(10), rnorm(10), rnorm(10), rnorm(10)),
  y   = c(rnorm(10), rnorm(10), rnorm(10), rnorm(10)),
  grp = rep(c("A","B","C","D"), each = 10)
)

p3 <- ggplot(df2, aes(x = x, y = y)) +
  geom_point(data = subset(df2, grp == "A"),
             colour = "blue",   size = 4,  shape = 16) +
  geom_point(data = subset(df2, grp == "B"),
             colour = "magenta", size = 4, shape = 2) +
  geom_point(data = subset(df2, grp == "C"),
             colour = "orange", size = 8,  shape = 17) +
  geom_point(data = subset(df2, grp == "D"),
             colour = "blue",   size = 6,  shape = 21, fill = "green") +
  xlim(-3, 3) + ylim(-3, 3) +
  theme_bw()

p1 | p2 | p3
```



Analysis example: regression

One of the most common type of data analysis is a regression model. Despite common and conceptually simple, it is a very powerful way to understand which (and how) of a number of candidate variables, sometimes referred to covariates, independent or explanatory variables, might influence a dependent variable, also often referred as the response. There are many flavors of regression models, from a simple linear regression to complicated generalized additive mixed models. We do not wish to present these in any detail, but to introduce you to some functions that implement these models and the syntax that R uses to describe them.

Let's start with the basics. You have used the `cars` data set above. We use it here again to try to explain the distance a car takes to stop as a function of its speed. We start with a linear model using function `lm`.

```
data(cars)
mylm1<-lm(dist~speed,data=cars)
```

We have stored the result of fitting the model in object `mylm1`. The function `summary` can be used to print a summary of the fit

```
summary(mylm1)
```

```
Call:
lm(formula = dist ~ speed, data = cars)

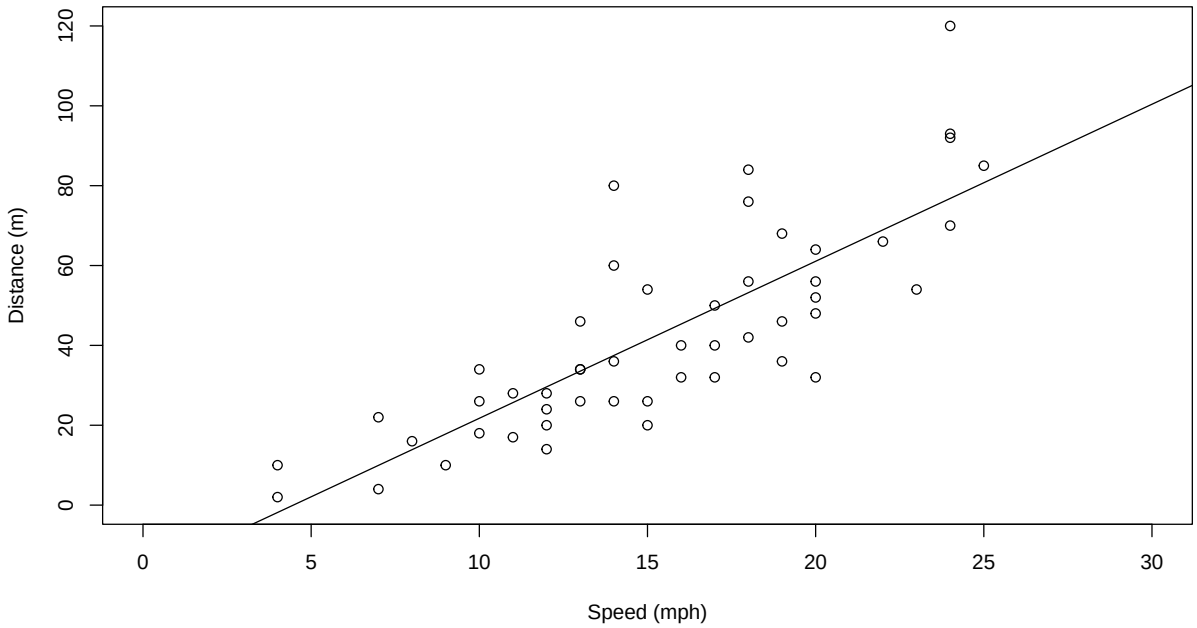
Residuals:
    Min      1Q  Median      3Q     Max
-29.069  -9.525  -2.272   9.215  43.201

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -17.5791     6.7584  -2.601   0.0123 *
speed         3.9324     0.4155   9.464 1.49e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 15.38 on 48 degrees of freedom
Multiple R-squared:  0.6511,    Adjusted R-squared:  0.6438
F-statistic: 89.57 on 1 and 48 DF,  p-value: 1.49e-12
```

Do not get frightened about all the output. The coefficient associated with speed tells us what intuition alone would anticipate, the higher the speed, the larger the distance a car takes to stop. The easier way to see the relationship is by adding a line to the plot (note this is a similar plot to what you should have created in task 3 above!). The predicted relationship is shown next:

```
xl<-"Speed (mph)"
yl<-"Distance (m)"
plot(cars$speed,cars$dist,xlab=xl,ylab=yl,ylim=c(0,120),xlim=c(0,30))
abline(mylm1)
```



Note how function `abline` is used with a linear model as its first argument and it uses the parameters in said object to add a line to the plot. The optional arguments `v` and `h` are often very useful to draw vertical and horizontal lines in plots.

Task 5: Use abline to draw dashed lines (tip, use optional argument `lty`=2) representing the estimated distance that a car moving at 16 mph would take to stop.

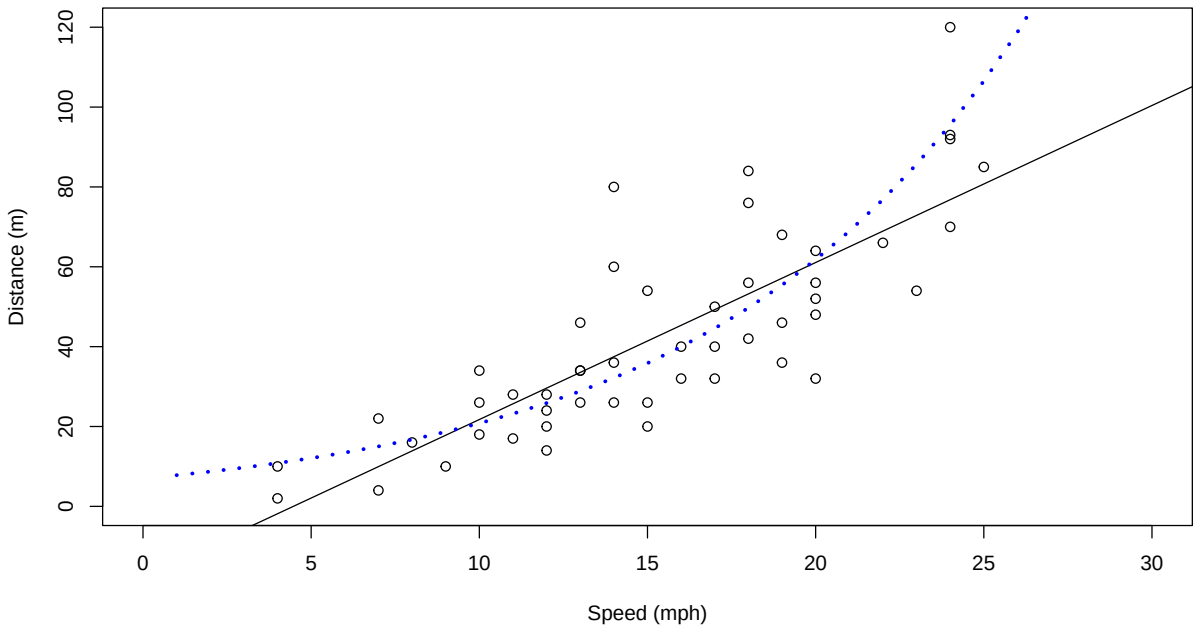
Note that the line added to the plot represents the distance a car would take to stop given its speed. Oddly enough, it seems like a car going at 3 mph might take a negative time to stop, which is just plain nonsense. Why? Because we used a model which does not respect the features of the data. A stopping distance can't be negative! However, implicit in the linear model we used, distance is a Gaussian (=normal) random variable. We can avoid this by using a generalized linear model (GLM). Now the response can have a range of distributions. An example of such distribution that takes only positive values is the gamma distribution. We implement a gamma GLM next

```
#fit the GLM
myglm1<-glm(dist~speed,data=cars,family=Gamma(link=log))
#predict using the GLM for speeds between 1 and 30
predmyglm1<-predict.glm(myglm1,
newdata=data.frame(speed=1:30),type="response")
```

Our model now assumes the response has a gamma distribution, and the link function is the logarithm. The link function allows you to change how the mean value is related to the covariates. This becomes rather technical rather fast. Details about GLMs are naturally beyond

the scope of this tutorial. References like Faraway (2006) or Zuur *et al.* (2009) will provide further details in an applied context. The predicted relationship is shown in the next figure.

```
#create a plot
plot(cars$speed,cars$dist,xlab="Speed (mph)",
ylab="Distance (m)",ylim=c(0,120),xlim=c(0,30))
#add the linear fit
abline(mylm1)
#and now add the glm predictions
lines(1:30,predmyglm1,col="blue",lwd=3,lty=3)
```



However, this GLM still requires that the response is linear at some scale (in this case, on the scale of the link function). Sometimes, non-linear effects are present. These can be fitted using generalized additive models. A good introduction to GAMs is provided by Wood (2006) and Zuur *et al.* (2009).

So finally we fit a GAM model to the same data set. For that we require library `mgcv`. The outcome is shown below. Here the fit is not very different from the GLM fit, but under many circumstances a GAM might be required over a GLM. We will see such an example in the next few days, when we model the detectability of beaked whale clicks as a function of distance and angle (with respect to hydrophones).

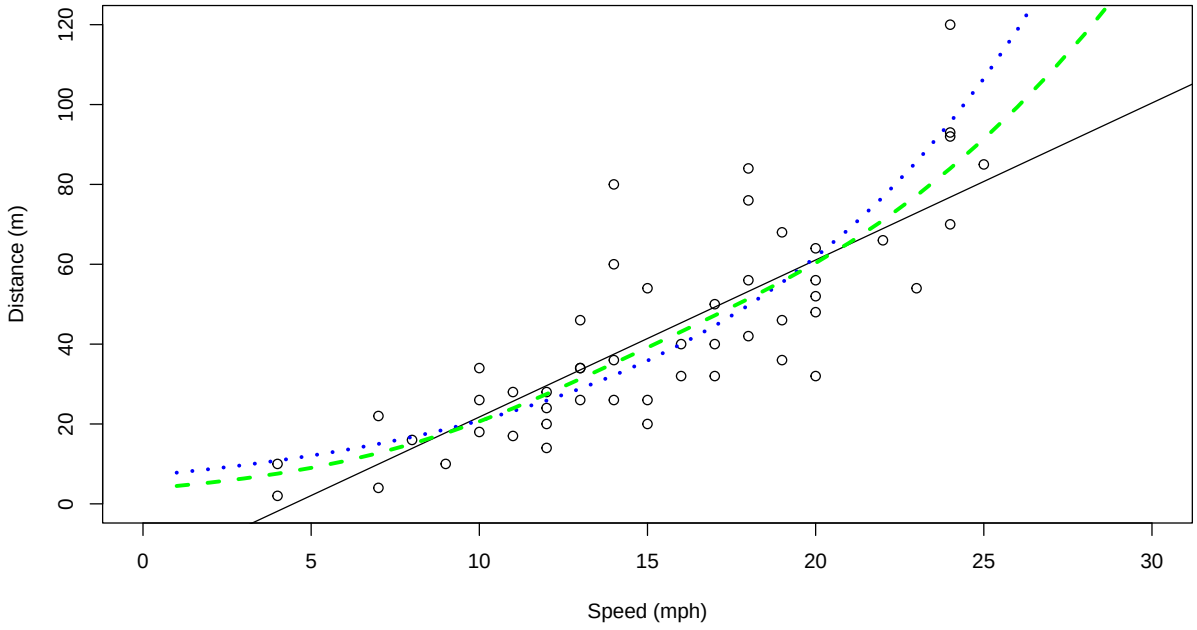
```
#load the mgcv library
library(mgcv)
```

```
Loading required package: nlme

This is mgcv 1.9-4. For overview type '?mgcv'.
```

```
#fit the GAM
mygam1<-gam(dist~s(speed),data=cars,family=Gamma(link=log))
#predict using the GAM for speeds between 1 and 30
predmygam1<-predict(mygam1,newdata=data.frame(speed=1:30),
type="response")
```

```
#create a plot
plot(cars$speed,cars$dist,xlab="Speed (mph)",
ylab="Distance (m)",ylim=c(0,120),xlim=c(0,30))
#add the linear fit
abline(mylm1)
#and now add the GLM predictions
lines(1:30,predmyglm1,col="blue",lwd=3,lty=3)
lines(1:30,predmygam1,col="green",lwd=3,lty=2)
```



Dates and Times

We might sometimes need to work with dates and/or times, and this used to be a major pain back in the day. Thanks to the functionality of package `lubridate`, that is no longer the

case.

```
#load the package
library(lubridate)
```

Dates and times are natively of class POSIXct, POSIXt in R. As an example, take a look at the current date and time using function `Sys.Date()`. It is now (as in, when this document was last compiled!)

```
#for just the date try
# Sys.Date()
dtnow <- Sys.time()
dtnow
```

```
[1] "2026-10-07 09:27:09 WEST"
```

```
class(dtnow)
```

```
[1] "POSIXct" "POSIXt"
```

and get just the date as

```
as.Date(dtnow)  # R grabs the date part automatically
```

```
[1] "2026-10-07"
```

We could get the day

```
day(dtnow)
```

```
[1] 7
```

or the Julian day

```
yday(dtnow)
```

```
[1] 280
```

or the minute

```
minute
```

```
function (x)
{
    UseMethod("minute")
}
<bytecode: 0x000002520cb5ca10>
<environment: namespace:lubridate>
```

```
(dtnow)
```

```
[1] "2026-10-07 09:27:09 WEST"
```

We could actually calculate the time elapsed between any time points

```
dtnow <- ymd_hms(dtnow, tz = "Europe/Lisbon")
Sys.time()-dtnow
```

```
Time difference of 0.06845999 secs
```

You can also turn strings (aka characters) into date-time objects, as an example, this is my birthday as text

```
bday <- mdy("July 22nd, 1975")
bday
```

```
[1] "1975-07-22"
```

and now this is is my age

```
Sys.Date()-bday
```

```
Time difference of 18705 days
```

a bit nicer and controling units, my age in years is

```r
as.numeric(interval(bday, Sys.Date()), "years")
```

```
[1] 51.2115
```

and as humans use age

```r
# Integer age (completed years)
interval(bday, Sys.Date()) %/% years(1)
```

```
[1] 51
```

Send me a present if it is my birthday!

Check out here the many other amazing things you can do with times and dates leveraging on `lubridate` functionality.

Simulation and random number generation

Another powerful use of R is for simulation. To this end, R has the ability to simulate random deviates from a large number of distributions. Perhaps the more useful and commonly used are the uniform and the Gaussian distributions. We now create 50 random deviates from each of these, as well as some Poisson deviates, for illustration

```r
#generate 50 pseudo-random Guassian numbers with mean 20 and standard deviation 3
rdnorm<-rnorm(50,mean=20,sd=3)
#generate 50 pseudo-random  50 uniform numbers between 3 and 6
rdunif<-runif(50,min=3,max=6)
#generate 50 pseudo-random  50 Poisson numbers with mean 6
rdpois<-rpois(50,lambda=6)
```

R can create random numbers from many different distributions (see help(Distributions) for a list) – the relevant functions generally start with r and then an abbreviated distribution name (`rbinom`, `rexp`, `rgeom`, etc). Additionally, R also includes the ability to obtain the density function, distribution function and quantile function via the `d`+name, `p`+name and `q`+name functions. As an example, the Gaussian function usage of these functions is presented below

```r
dnorm(0,mean=0,sd=1)
```

```
[1] 0.3989423
```

```r
pnorm(0,mean=0,sd=1)
```

```
[1] 0.5
```

```r
qnorm(0.975,mean=0,sd=1)
```

```
[1] 1.959964
```

Task 6: Using what you have learnt here, create two histograms, one of 50, another of 5000, random deviates from a Gaussian distribution (you can choose the mean and standard deviation you prefer!), using the optional argument `freq=FALSE` (leading to an estimate of the density function). Then add a line to the plot that represents the true underlying density (tip, you can use function `dnorm`), and comment on the results. You can also do similar experiments with other distributions. How weird are a beta(1,1), a beta(1,5) and a beta(0.5,0.5) distributions (tip, you can use function `rbeta` or `dbeta`). Can you guess which one is sometimes referred to as bath tub distribution. What quantities do you imagine a beta might be useful to model?

Programming tricks

Some very useful programming structures are those required to evaluate conditional statements and those used to repeat statements many times. These are fundamental for implementing simulations. In R we have `if` statements and `for` loops, respectively.

As an example, see how an if statement works

```r
X=2
if (X>0) print(X+3)
```

```
[1] 5
```

One can also use an if-else statement, which executes either (1) something or (2) something else, depending on the condition being TRUE or FALSE. Here's an example:

```r
X=2
if (X>0)
  {Y=abs(X)} else
  {Y=X^2}
Y
```

```
[1] 2
```

```
X=-5
if (X>0)
  {Y=abs(X)} else
  {Y=X^2}
Y
```

```
[1] 25
```

on the other side, here's how a for loop works

```
n=4
X=1:n
for (i in 1:n) print(i+3)
```

```
[1] 4
[1] 5
[1] 6
[1] 7
```

note there is nothing special about the use of i for an index; you can use any index that you might want

```
n=4
X=1:n
for (j in 1:n) print(sum(c(j,j+3)))
```

```
[1] 5
[1] 7
[1] 9
[1] 11
```

or even

```
n=4
X=1:n
for (i in X) {
  cat(paste("The i currently is:",i),sep="\n")
  cat(paste("The i+3 currently is:",i+3),sep="\n")
  }
```

```
The i currently is: 1
The i+3 currently is: 4
The i currently is: 2
The i+3 currently is: 5
The i currently is: 3
The i+3 currently is: 6
The i currently is: 4
The i+3 currently is: 7
```

See above, explore R. Change the code. Repeat. Check for yourself what `cat` and `paste` can be used for!

Task 7: Create 9 histograms of samples of Gaussian random variables, adding the mean value on the plot as a vertical dashed line, in blue if the mean of the observations is positive and in red if the mean of the observations is negative.

Other interesting structures for "control flow" are the `while`, `repeat` and `break`. Look into the help, `?if`, to see details.

Writing your own functions

While the above functions, and the many more available, make R a very useful tool, there are sometimes problems which require a special tool. For these, we can create our own functions. Note this is an advanced topic.

The way of doing that follows a specific syntax

```
> name <- function(arg1,arg2,...) {what the function does goes here}
```

As an example, we create a function that returns the sum of its 2 arguments:

```
myfun<-function(i,j){
  myres<-i+j
  return(myres)
}
```

You can now see the function in the works

```
myfun(3,5)
```

```
[1] 8
```

Note a function could have many arguments, none, or just one.

Task 8: create a function called `mystats` which returns the mean, variance, maximum and minimum of the first, and only, argument (a vector). Then, update your function such that it can also return the mean excluding the negative numbers. Then, create some other function you might think could be useful.

Creating your own functions will unleash strong R power, increasing significantly your ability to manipulate, analyze and simulate data.

Final tasks

These tasks are only intended for the students in Modelação Ecológica. If you are a Ecologia Numérica student you can try it at your own risk! If you are neither of those, then feel free to work through the material regardless.

A final task integrating all of the above

Here we will implement an exercise were we pretend we are sampling an animal population, using some (very basic) simulations to understand the process better. Create plots that represent all the steps of your task, with proper legends, labels, colors, etc, and add all your comments to the dynamic report.

This exercise simulates a distance sampling survey. If you want to know more about it, you can check this 2 page introduction paper on the topic here. It is one of the most often used methods to estimate the abundance and/or density of wildlife populations.

1. Simulate the positions of 10000 animals in a study area, with length 10km and width 1km. Assume that any animal has an equal chance to be at any location in the study area (this corresponds to a uniform density surface).
2. Generate a transect at a random location along the study area.
3. Assume that you can potentially detect at most animals up to 500 meters from your transect. Count all the animals that you would detect if detection was perfect across your transect.
4. Consider that animals far from your transect are harder to detect - yes, you are doing distance sampling! Define a function that represents a distance sampling half-normal detection function. If you do not know what that looks like check here, around slide 9. Assume that sigma=200m.

5. Simulate the detection process and get a sample of those animals detected. This is the hard bit, creating an animal detector. Tip: using `runif` will help. Ask me for details if you need them.
6. Create a plot that allows you to estimate (at this stage just a visual guess is needed) the detection probability.
7. Repeat the sampling process 500 times, and store the number of animals detected in each one of your simulated surveys.
8. plot the distribution of the number of animals that you would detect each new survey.

Take your own conclusions about all that you did.

Creating your own statistical test via resampling

This is work in progress

We are all used to implementing statistical tests. But statistical tests for the situations one might face with real ecological data are not always available. For that reason, it might be useful to know how to create your own statistical tests. One can do that if one is able to create a test statistic, that contains information about the hypothesis one wishes to test, and for which we can simulate the distribution assuming that the null hypothesis is true.

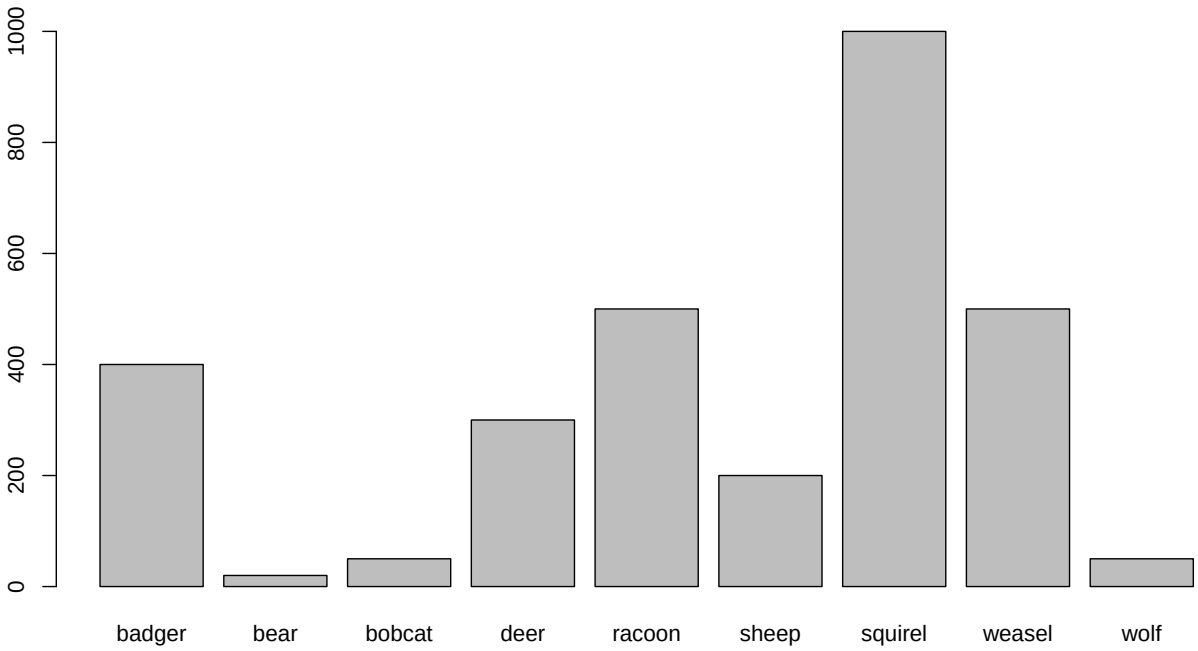
Let us consider an example where we have data from camera traps, and we want to see if there is some relationship between two species. In particular, we want to see if the use of the area covered by the camera is dependent on what species were present there before.

Imagine we have a camera trap and we collect the order in which different species appeared on the camera.

```
#the exercise is more interesting if you do not think about how the data is being created
#later you can look at it, but for now, just treat this as a black box
#so all get the same answers
set.seed(189)
species<-c("wolf","bear","deer","racoon","weasel","bobcat","badger","squirel","sheep")
abundances<-c(50,20,300,500,500,50,400,1000,200)
appearences<-rep(species,times=abundances)
n<-length(appearences)
appearences<-appearences[sample(1:n,size = n,replace=FALSE)]
```

The code above has simulated such data, creating 3020 sequential occurrences of 9 different species. Here's the number of observations per species

```
barplot(table(appearences))
```



We can see the first few detections

```
head(appearences)
```

```
[1] "weasel"  "squirel" "badger"  "racoon"  "squirel" "sheep"
```

Recent research indicates that squirels tend to appear after weasels appear, and you have been tasked with looking into the data to test whether that seems to be the case at this site. How can you do it?

You will immediately realize this puts you in a context that you have not learnt any statistical significance tests that sound useful here. You need to create your own test. This can be done as a resampling/permutation test.

You first define a null hypothesis. That could be

H0: the appearance of a squirrel does not depend on badgers having been present

Next, you need to get a test statistic. It seems like a good test statistic would be the number of times a squirrel is preceded by a badger.

How can we calculate that for our data?

```r
#get each appearence
firsta<-appearences[1:(n-1)]
#get the next one
seconda<-appearences[2:(n)]
#get the pairs
paireda<-data.frame(fa=firsta,sa=seconda,pa=paste(firsta,seconda,sep="."))
#now, it's easy to count how many squirel-badger sequences we had
test.stat<-sum(paireda$pa=="squirel.badger")
test.stat
```

```
[1] 131
```

The key question is, under H0, is the value `test.stat` extreme, leading to rejecting H0, or quite expected, leading to no evidence to reject H0?

What do you think? How can you find a distribution for the test statistic under H0 so that you can formally evaluate whether to reject, or not, H0.

Wrap up

A full introduction to R course could take an entire week. A full course in regression modelling with R could take an entire semester. A full course of data analysis in R could take a lifetime.

Our objective with this tutorial was simply to introduce you to R such that when we use R in the next few days/weeks, for whatever course you might be taking, the commands do not look too esoteric. Nonetheless, this material, as well as the references provided, should constitute a good basis to learn R further if you so desire.

Beginners find the R learning curve is often steep, but once mastered, R simplifies enormously the task of statistical data analysis.

Finally, to promote good habits, we clean the workspace. An organized workspace is very important!

```r
#cleaning the workspace
rm(list = ls())
```

Acknowledgements

Several people have provided comments over the years, typically when exposed to the tutorial, including colleagues, folks that have used it as teachers, or students asking questions in a course using the tutorial, like R courses, PAM DE courses, and Modelação Ecológica and Ecologia Numérica courses at DBA, FCUL, or doing internships / being supervised by me. I thank their kind contributions here: Danielle Harris, Len Thomas, Soraia Pereira, Susana França, Sofia Reboleira, Sónia Coelho, Afonso Barrocal, José Pedro Granadeiro, Daniela Leal, Nuno Escamade and Theoni Photopoulou. Many are not named explicitly, as I've forgotten about the specifics, but I thank them any way! If you think your name is missing, let me know, and I will add it here. Further, this list is ever evolving and so, if you have comments, please, send them my way and get your name added to it! Indeed, fame for eternity is indeed that close :)

References

Faraway, J.J. (2006). *Extending the linear model with R.* Chapman & Hall / CRC.

Wood, S.N. (2006). *Generalized additive models: An introduction with R.* CRC/Chapman & Hall.

Zuur, A.F., Ieno, E.N., Walker, N., Saveliev, A.A. & Smith, G.M. (2009). *Mixed effects models and extensions in ecology with R.* Springer.